CNNs 的优化

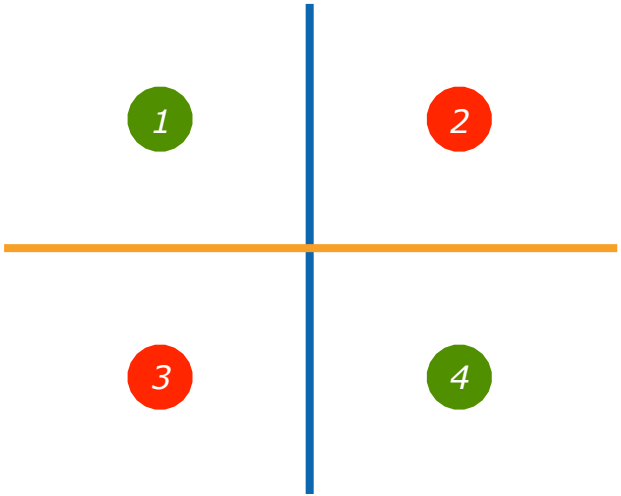
讲师：Hao Wang
AI Thrust，信息枢纽



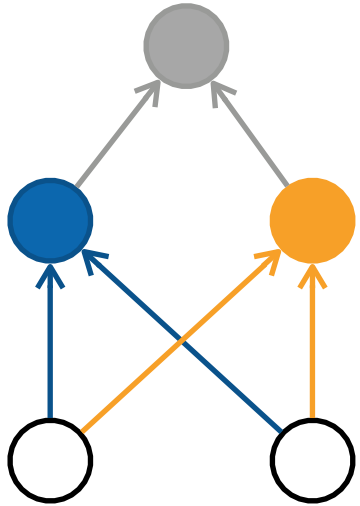
上周回顾



学习 XOR



	2	3	4	6
	+	-	+	-
	+	+	-	-
乘积	+	-	-	+



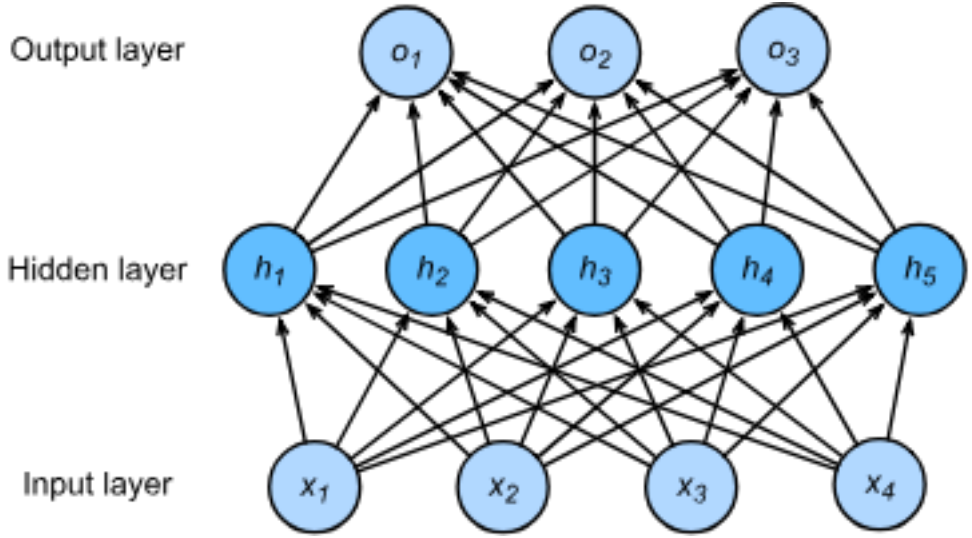
单隐藏层

- 输入 $\mathbf{x} \in \mathbb{R}^n$
- 隐藏 $\mathbf{W}_1 \in \mathbb{R}^{m \times n}$, $\mathbf{b}_1 \in \mathbb{R}^m$
- 输出 $\mathbf{W}_2 \in \mathbb{R}^m$, $\mathbf{b}_2 \in \mathbb{R}$

$$\mathbf{h} = \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1)$$

$$\mathbf{o} = \mathbf{w}^T \mathbf{h} + b_2$$

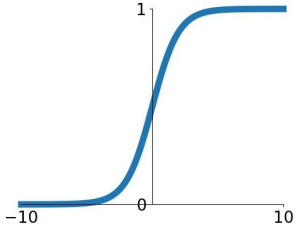
σ 是一种逐元素激活函数



激活函数

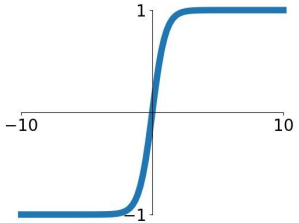
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



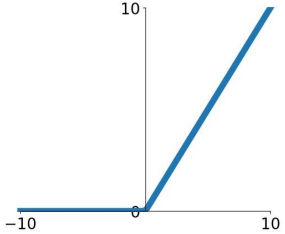
□□□□

$$\tanh(x)$$



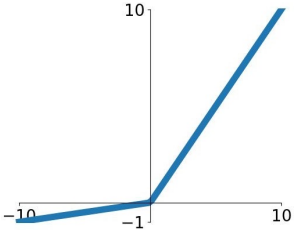
h □a □

$$\max(0, x)$$



泄漏型 ReLU

$$\max(0.1x, x)$$

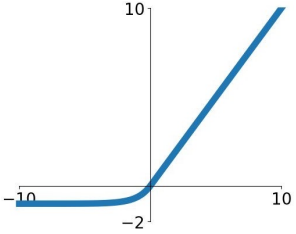


Maxout

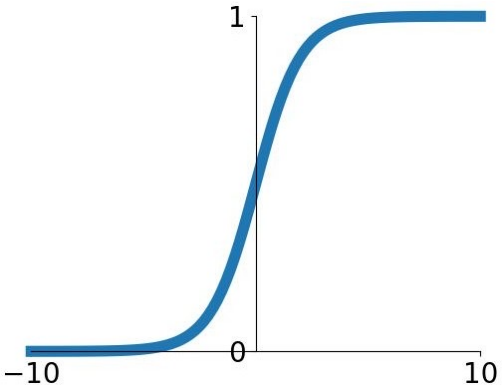
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

Ra □

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



激活函数



Sigmoid

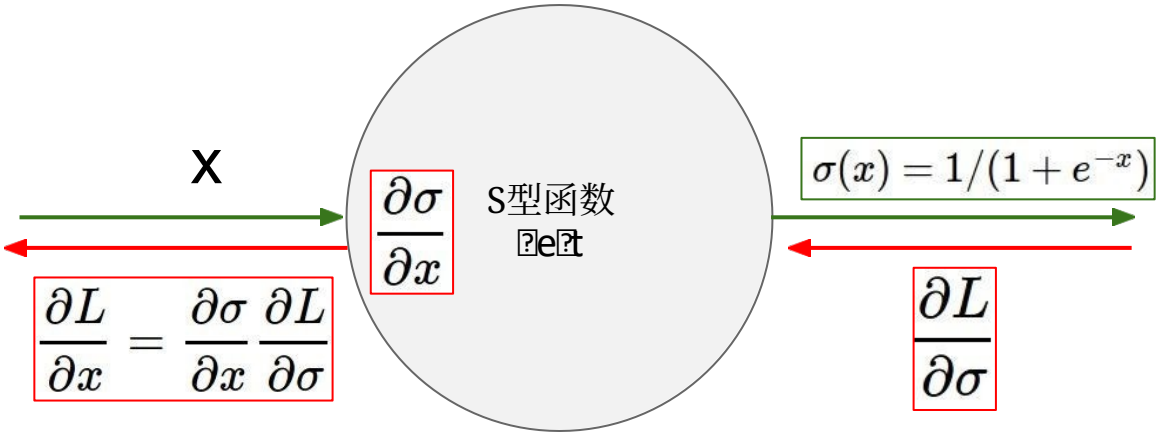
$$\sigma(x) = 1/(1+e^{-x})$$

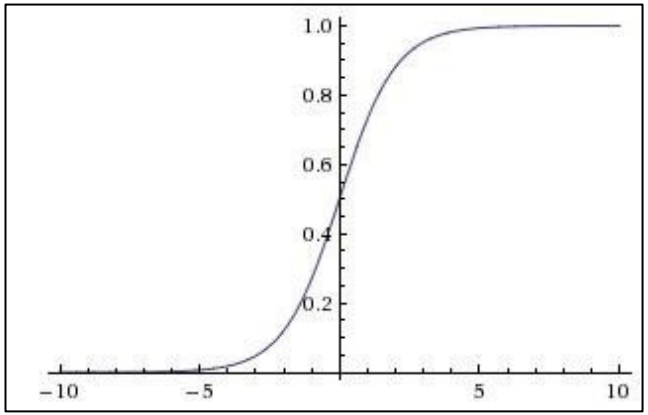
- 将数值压缩到范围 [0,1]
- 历史上很受欢迎，因为它们可以很好地解释为神经元的饱和“放电率”

1. 饱和神经元会“杀死”梯度

2. Sigmoid 输出不是零中心的

为什么这不好?





当 x =-10 时会发生什么？当 x = 0 时会发生什么？ 当 x = 10 时会发生什么?

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x))$$

$$\sigma(x) = \sim 1 \qquad \frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x)) = 1(1-1) = 0$$

如何解决“饱和神经元”问题?

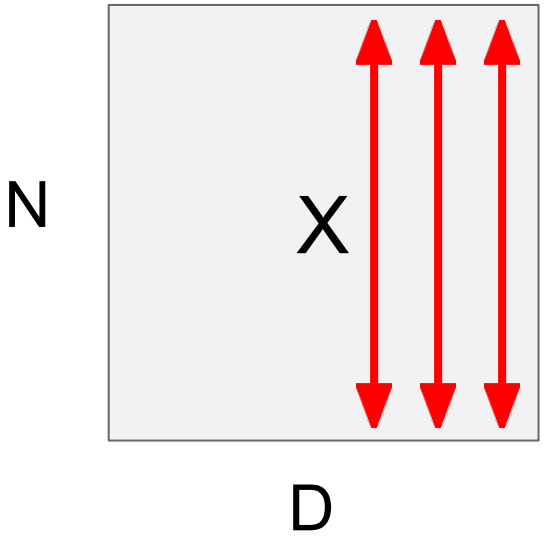
- 我们需要避免特征值过大或过小

➢ **归一化**!



批归一化

输入v $x : N \times D$



$$\mu_j = \frac{1}{N}\sum_{i=1}^{N} x_{i,j}$$

逐通道均值，
形状为 D

$$\sigma_j^2 = \frac{1}{N}\sum_{i=1}^{N}(x_{i,j} - \mu_j)^2$$

逐通道方差，
形状为 D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

归一化后的 x，
形状为 N x D

问题：如果零均值、单位方差这一约束过于苛刻，该怎么办？

批量归一化

输入v $x : N \times D$

可学习的缩放和平移参数:

$\gamma, \beta : D$

学习= ，γ σ β μ 将恢复恒等函数!

$$\mu_j = \frac{1}{N}\sum_{i=1}^{N} x_{i,j}$$

逐通道均值,
形状为 D

$$\sigma_j^2 = \frac{1}{N}\sum_{i=1}^{N}(x_{i,j} - \mu_j)^2$$

Per-channel var,
shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

归一化后的 x ,
形状为 N x D

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

输出,
形状为 N x D

[Ioffe 和 Szegedy， 2015]



批量归一化：测试时

估计值依赖于小批量；在测试时无法这样做！

输入v $x : N \times D$

可学习的缩放和平移参数：

$$\gamma, \beta : D$$

学习= 、γ σ β μ 将恢复恒等函数！

$$\mu_j = \frac{1}{N}\sum_{i=1}^{N} x_{i,j}$$ 逐通道均值，形状为 D

$$\sigma_j^2 = \frac{1}{N}\sum_{i=1}^{N}(x_{i,j} - \mu_j)^2$$ 逐通道方差，形状为 D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$ 归一化 x ，形状为 N x D

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$ 输出，形状为 N x D

批量归一化：测试时

输入v $x : N \times D$

可学习的缩放和平移参数：

$\gamma, \beta : D$

在测试期间，批归一化会变成线性算子！可以与其之前的全连接层或卷积层融合

$$\mu_j = \text{训练期间所见数值的（滑动）平均值}$$

逐通道均值，形状为 D

$$\sigma_j^2 = \text{训练期间所见数值的（滑动）平均值}$$

逐通道方差，形状为 D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

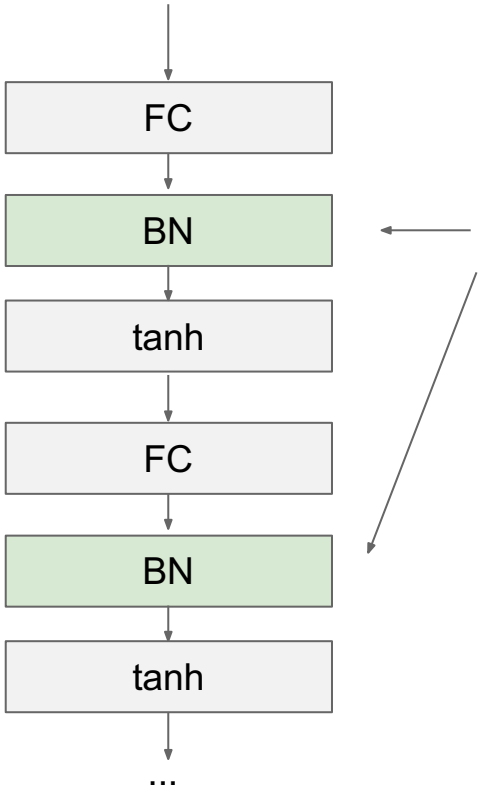
归一化后的 x，形状为 N x D

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

输出，形状为 N x D



批量归一化



通常插入在全连接层或卷积层之后、非线性激活之前。

$$\widehat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}]}}$$

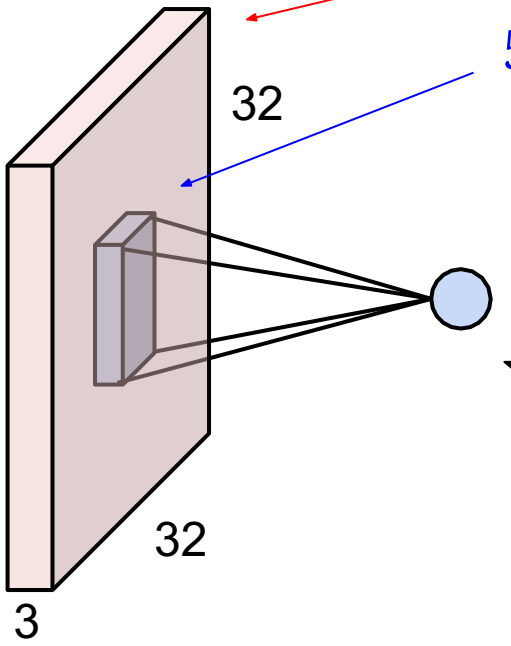
[Ioffe 和 Szegedy， 2015]

我们如何确保模型优化良好?

- （上周）非饱和激活函数
- 架构
- 优化器
- 正则化



卷积层



32x32x3 图像

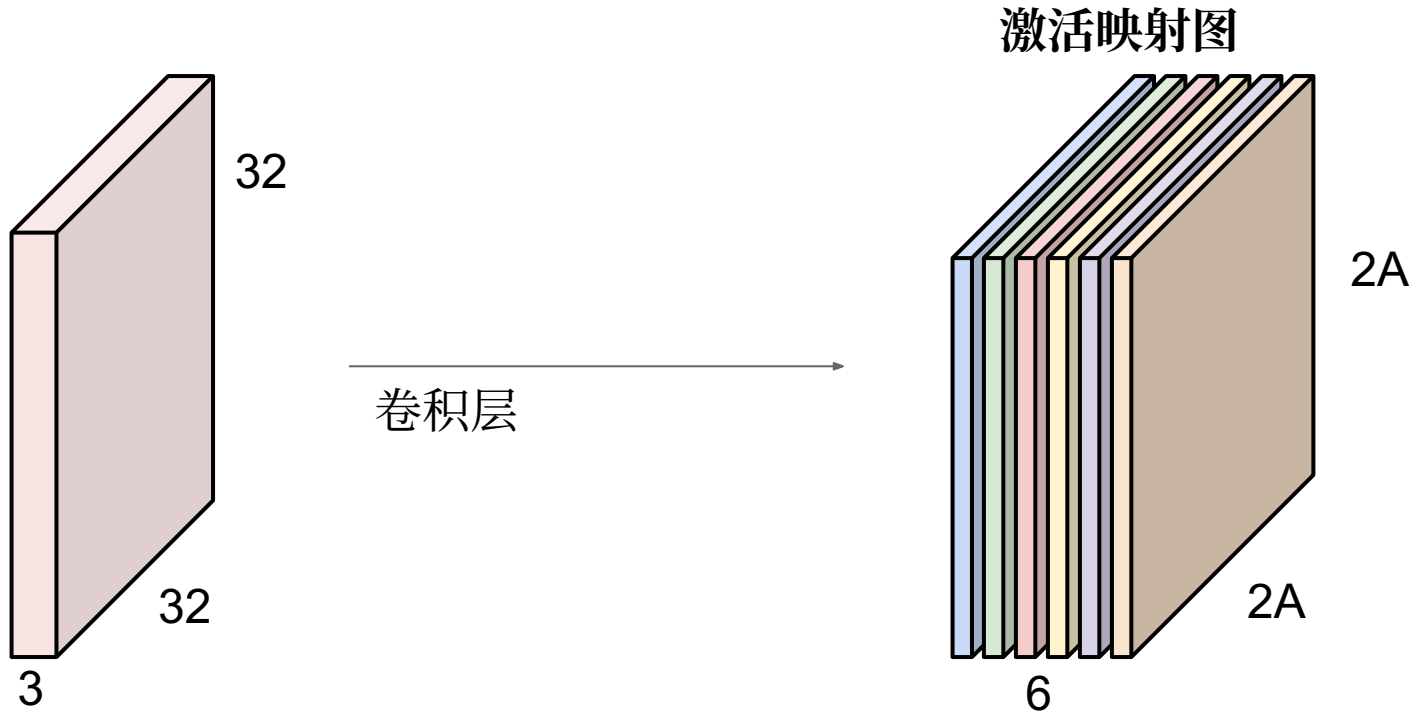
5x5x3 滤波器w

1 个数字：

将滤波器与图像中一个小的 5x5x3 区块进行点积的结果（即 5*5*3 = 75 维点积 +偏置）

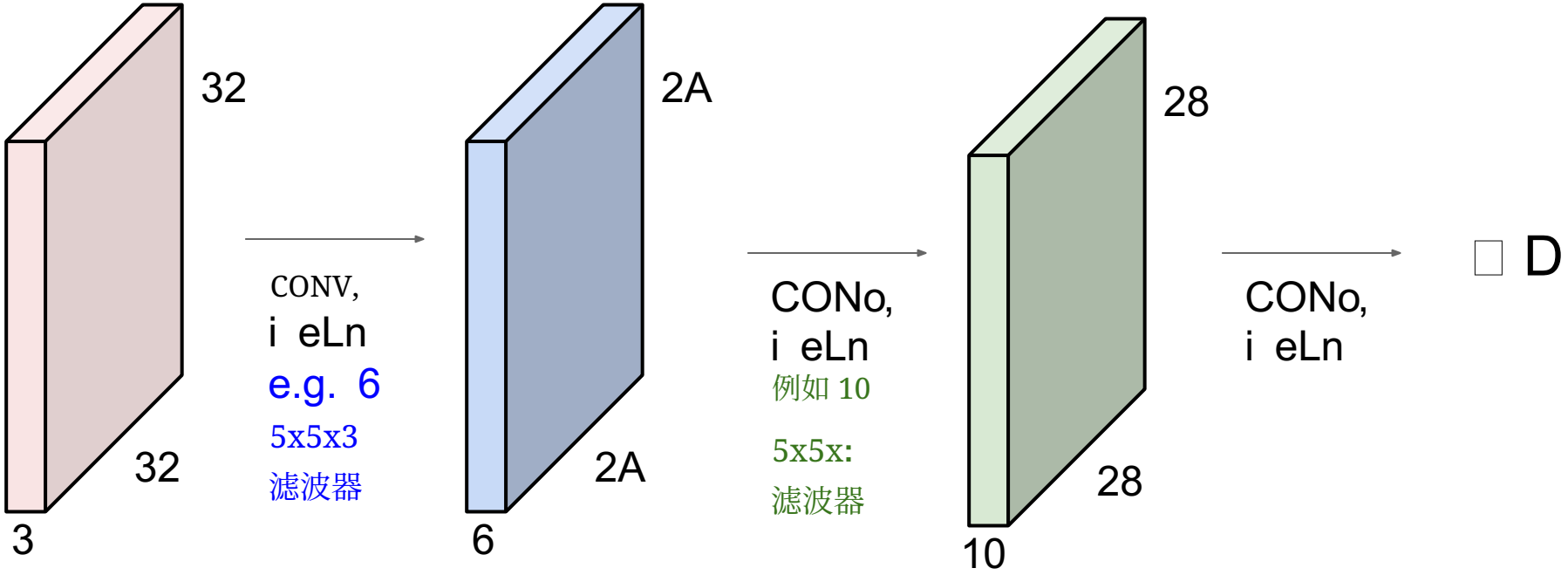
$$w^T x + b$$

例如，如果我们有 6 个 5x5 滤波器，就会得到 6 个单独的激活映射图：



- 我们将这些堆叠起来，得到一张尺寸为 28x28x6 的“新图像”！

ConvNet 是一系列卷积层，中间穿插着激活函数

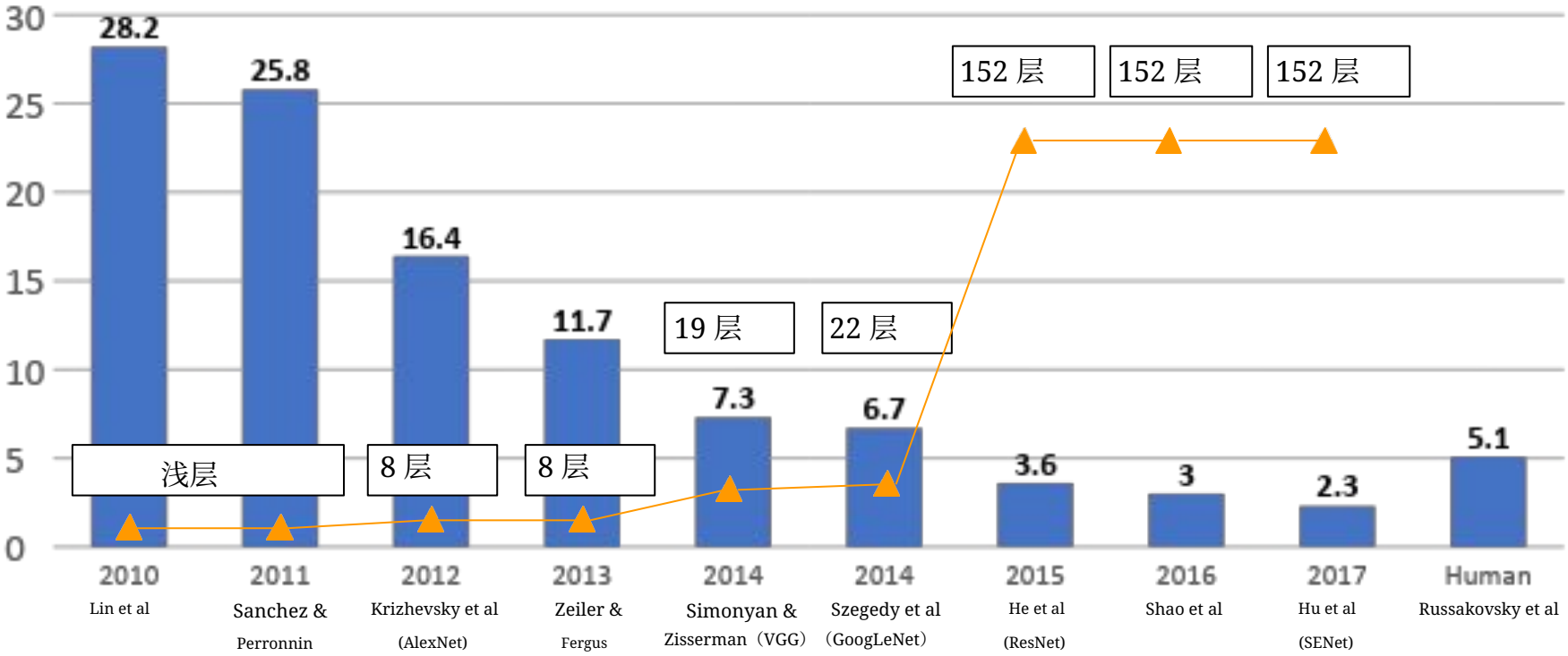


我们如何确保模型优化良好?

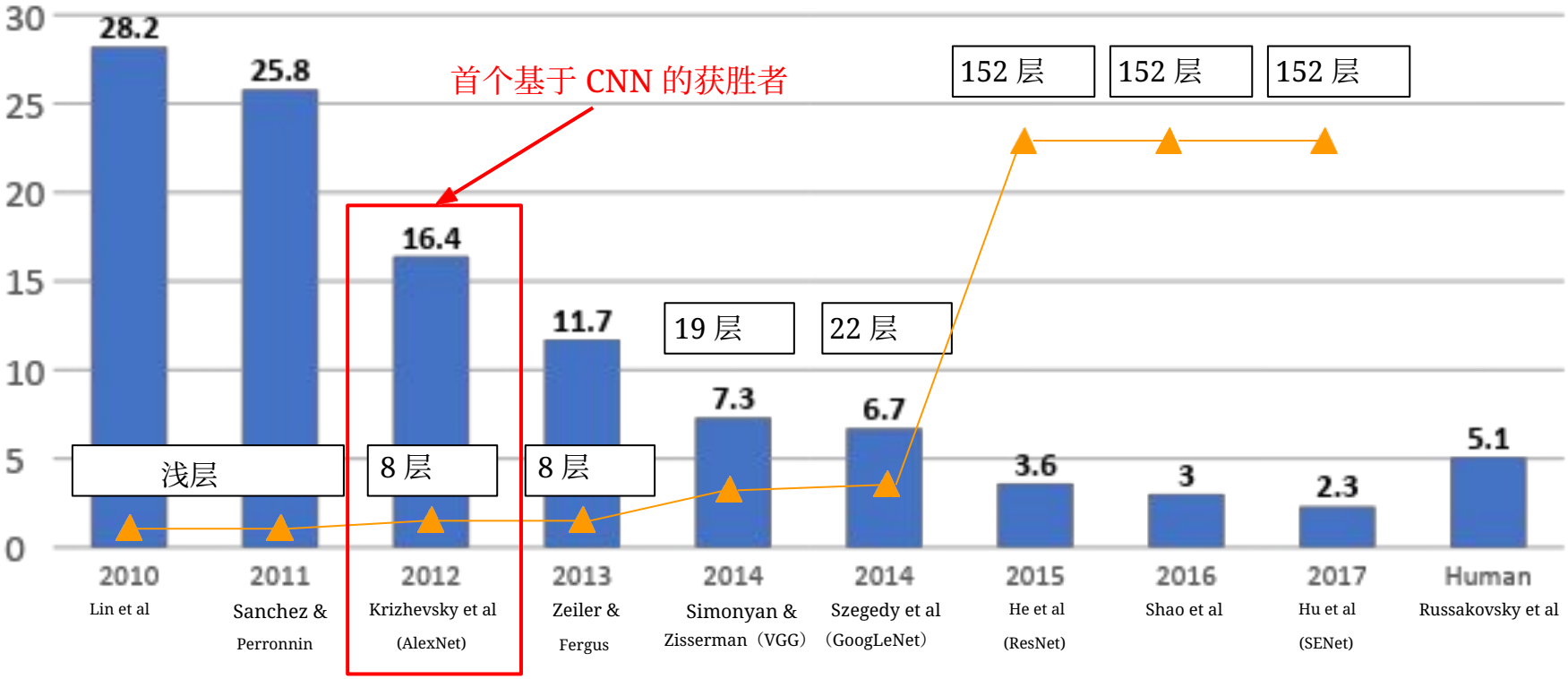
- （上周）非饱和激活函数
- 架构
- 优化器
- 正则化



ImageNet大规模视觉识别挑战赛（ILSVRC）获胜者



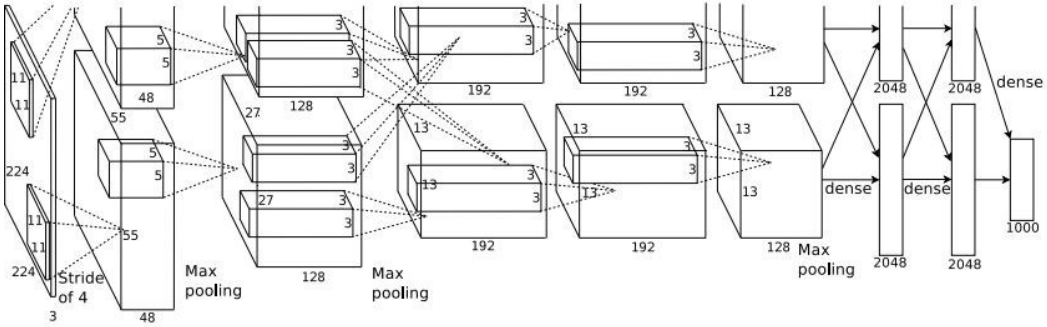
ImageNet 大规模视觉识别挑战赛（ILSVRC）获胜者



案例研究：AlexNet

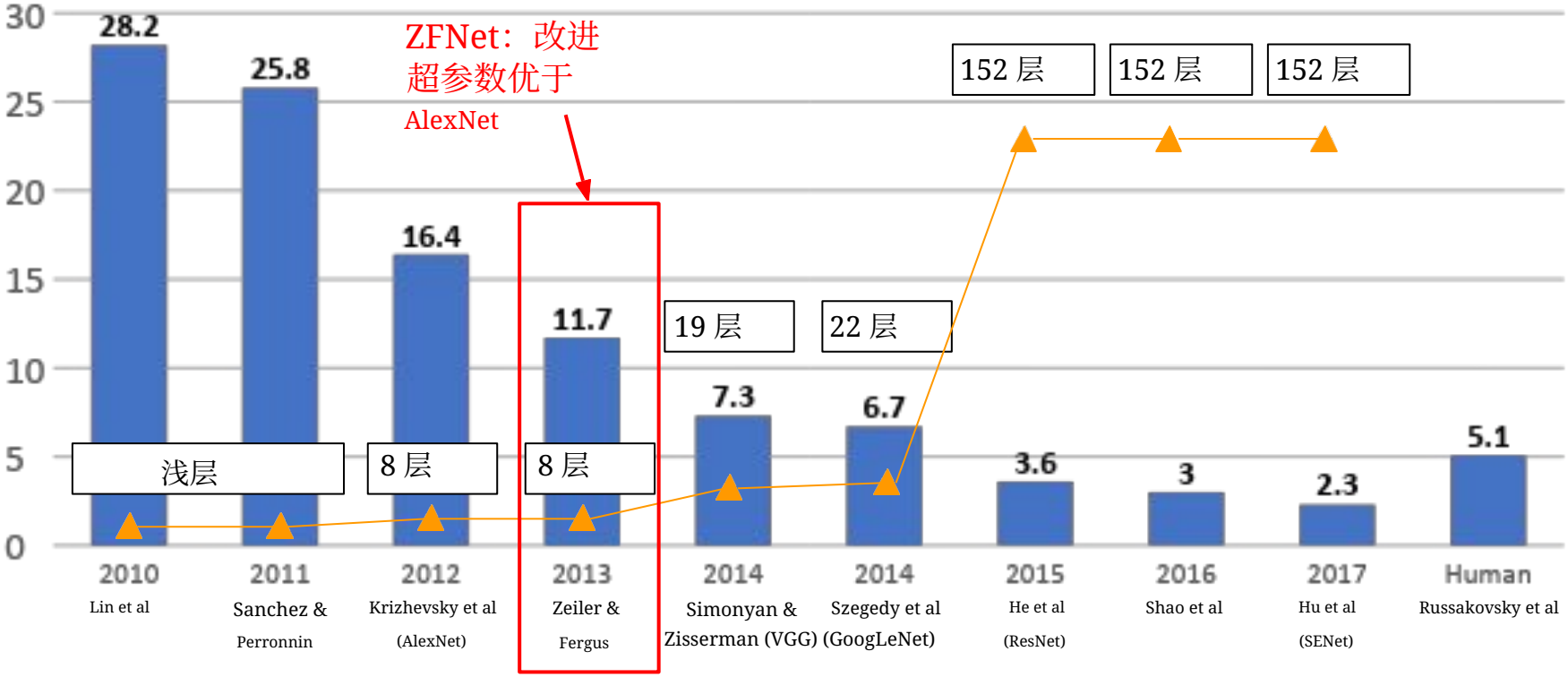
Krizhevsky et al. 5245

架构：CONV1MAX
P
O
O
L
1
NORM1CONV2MAX
P
O
O
L
2
N
O
R
M
2
C
O
N
V
3C
ONV4CONV7Ma
POOL3FC8FC9FC=

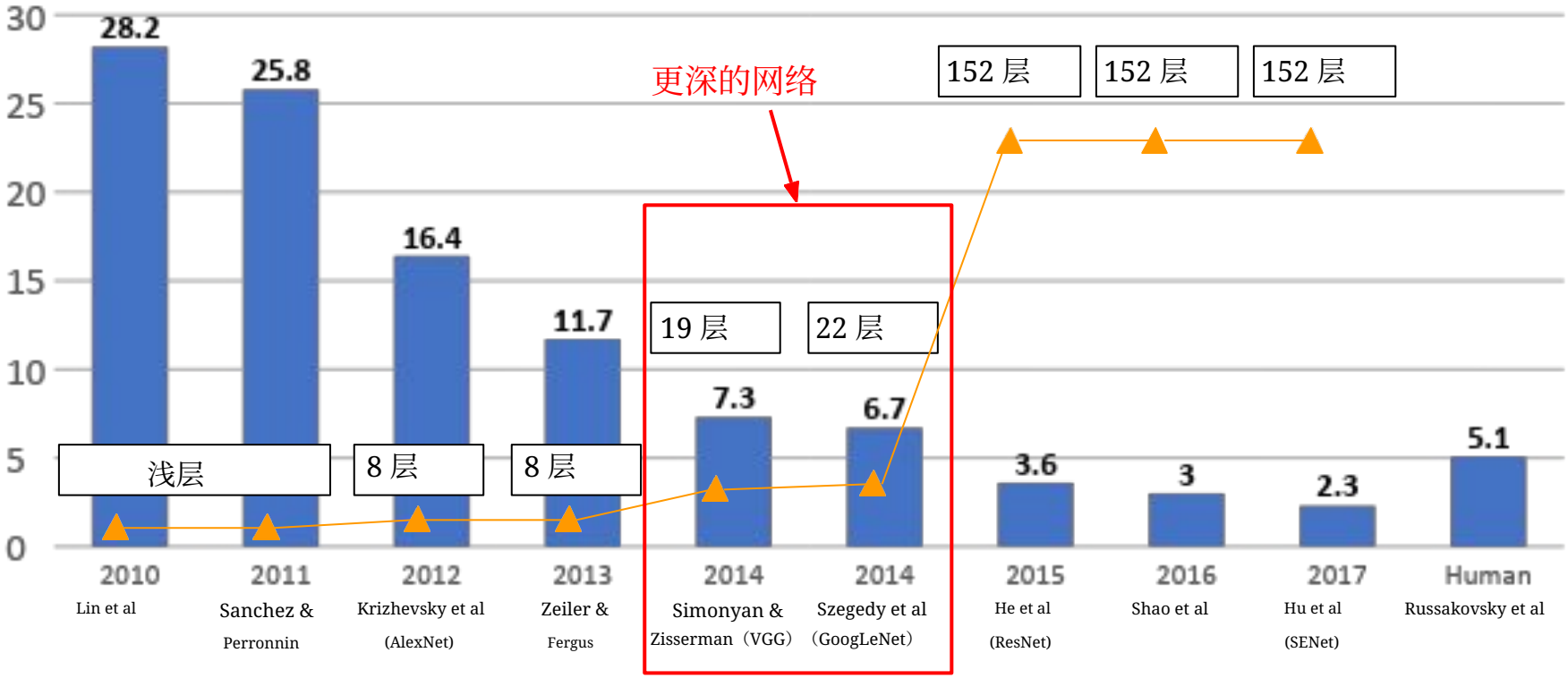


图片版权归 Alex Krizhevsky、Ilya Sutskever 和 Geoffrey Hinton 所有，2012。经许可转载。

ImageNet 大规模视觉识别挑战赛（ILSVRC）获胜者



ImageNet大规模视觉识别挑战赛（ILSVRC）获胜者



案例研究：VGGNet

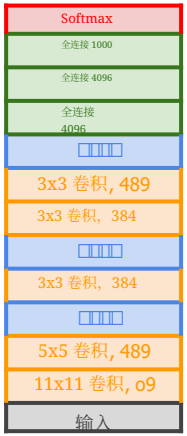
□*Simonyan* 和 *Zisserman*， *524[* □

问：为什么使用更小的滤波器？（3x3 卷积）

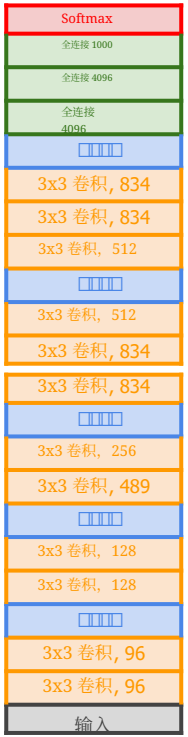
问：三个 3x3 卷积（步幅 1）层的有效感受野是什么？

三个 3x3 卷积（步幅 1）层堆叠起来，
其 **有效感受野** 与一个 7x7 卷积层相同

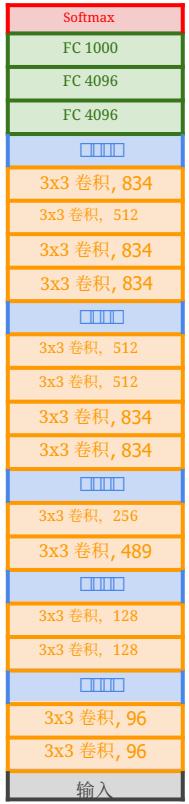
Wh□□



AlexNet

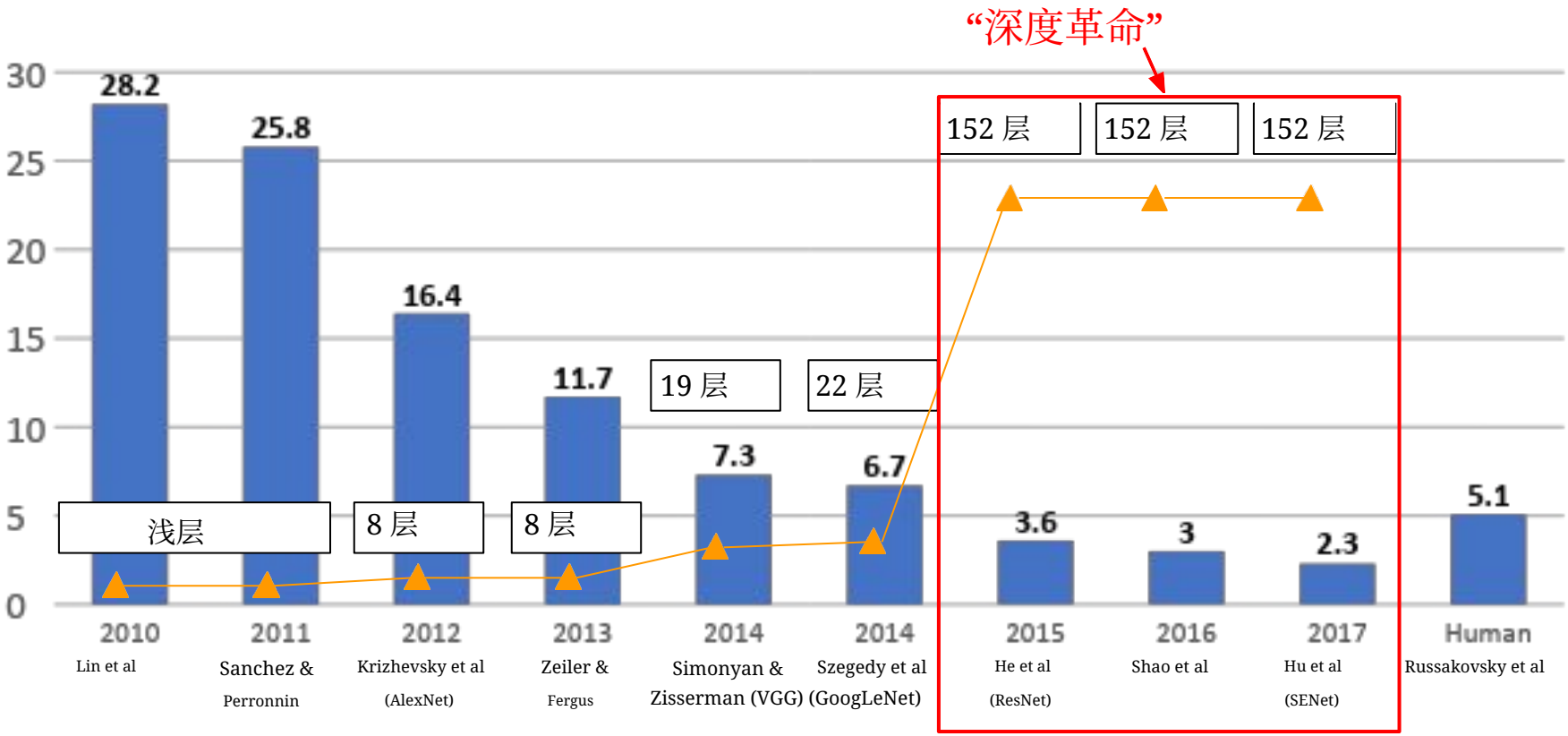


VGG16



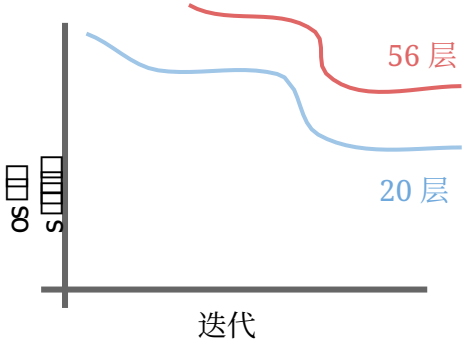
VGG19

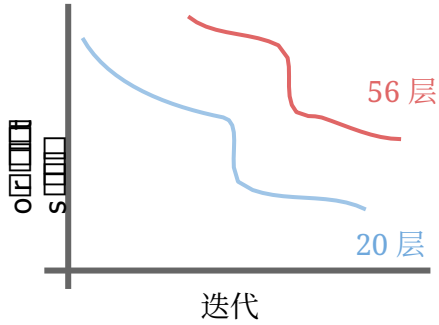
ImageNet大规模视觉识别挑战赛（ILSVRC）获胜者



如果我们再深入一些会怎样?

当我们在“普通”卷积神经网络上继续堆叠更深的层时，会发生什么?

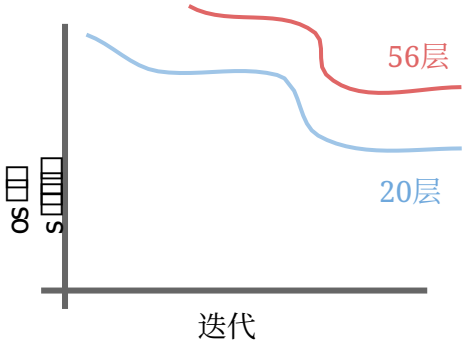


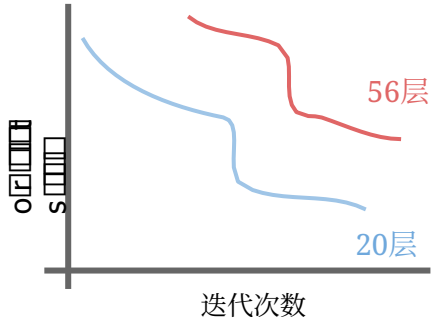


如果我们再深一些呢?

过拟合?

当我们在“普通”卷积神经网络上继续堆叠更深的层时，会发生什么?





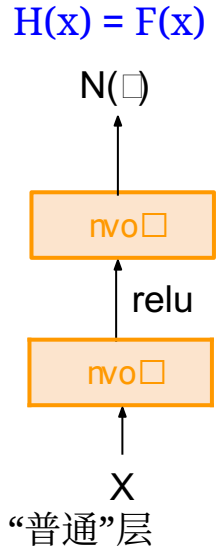
56层模型在测试误差和训练误差上都表现更差 -M更深的模型表现更差，

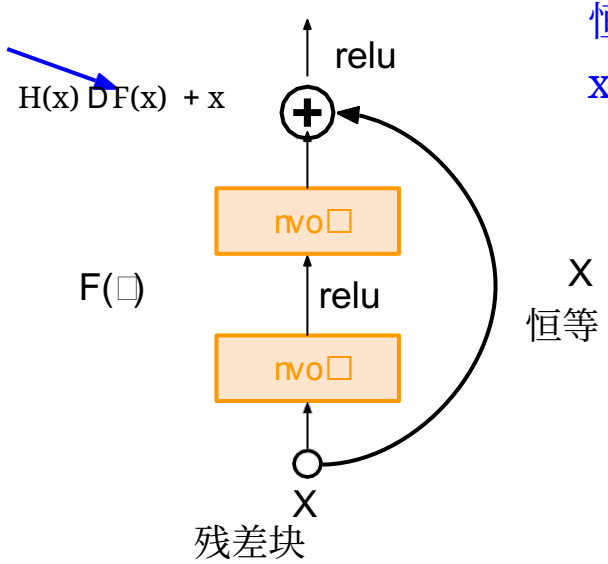
但这并不是由过拟合导致的！ -M退化问题

ResNet

[He et al., 524]

解决方案：使用网络层来拟合残差映射，而不是直接尝试拟合期望的底层映射





恒等映射：H(x) = x 若 F(x) = 0

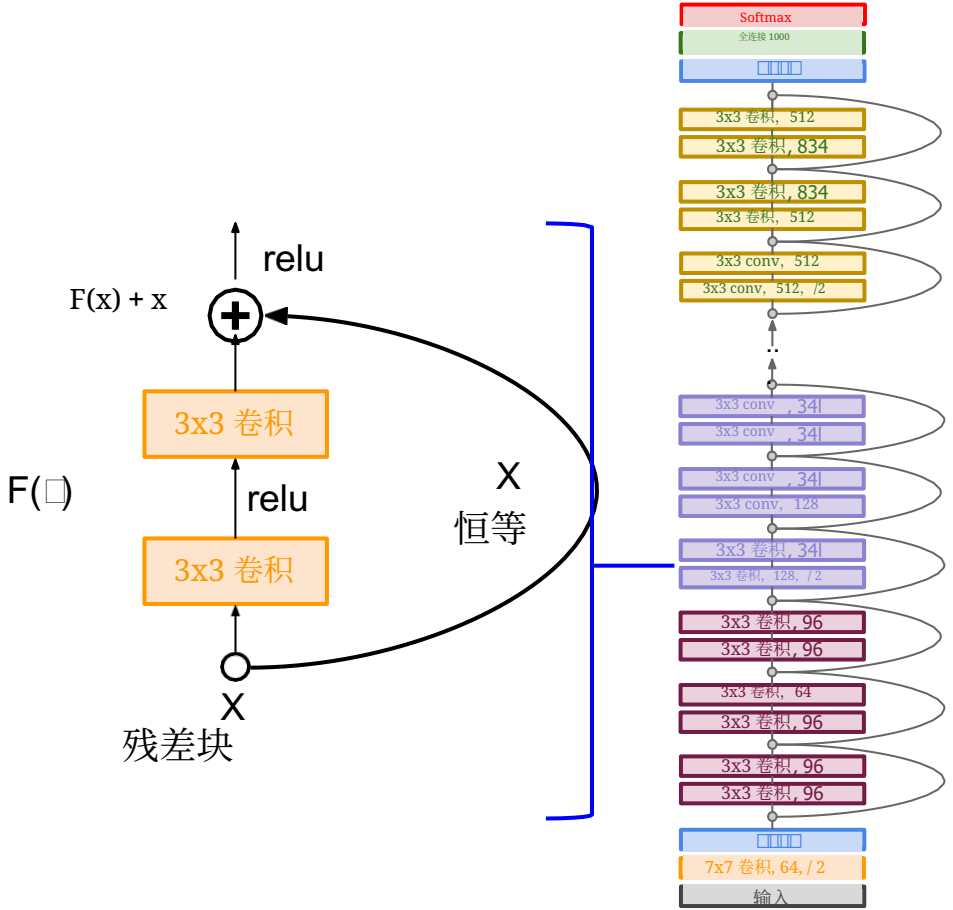
使用层来拟合 **残差F(x) = H(x) - x**，而不是直接拟合 H(x)

ResNet

□*He et al., 524]* □

完整 ResNet 架构：

- 堆叠残差块 - 每个残差块有两个 3x3 卷积层

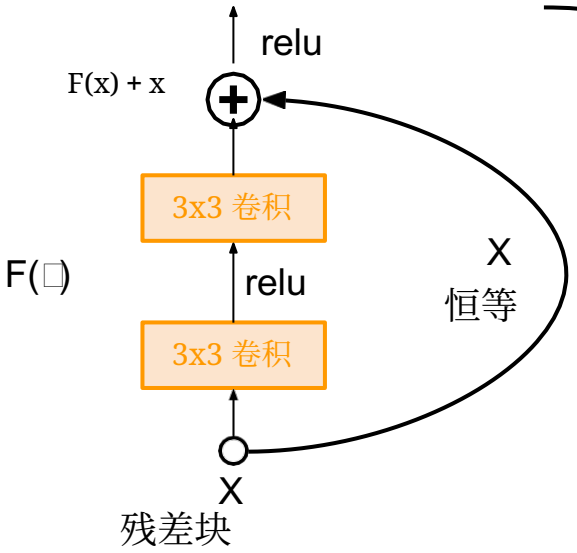


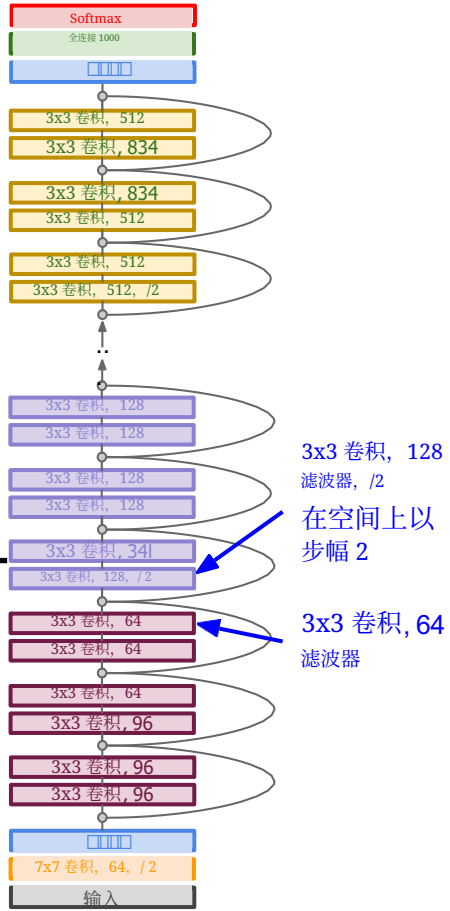
ResNet

□*He et al., 524]* □

完整 ResNet 架构：

- 堆叠残差块 - 每个残差块都包含两个 3x3 卷积层 - 定期将滤波器数量加倍，并使用步幅 2 在空间上降采样（每个维度 /2）



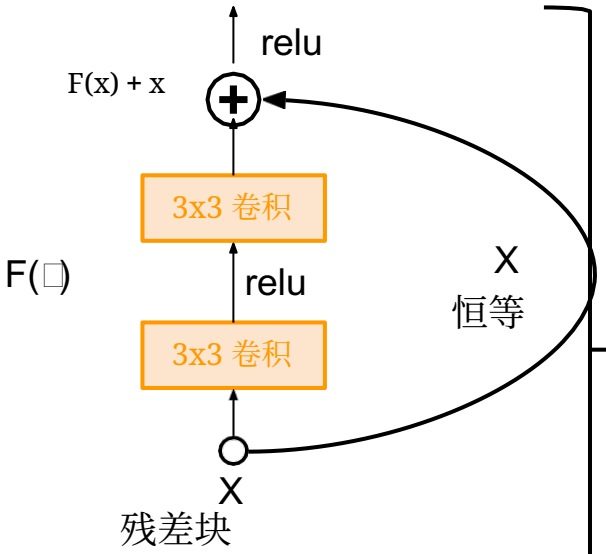


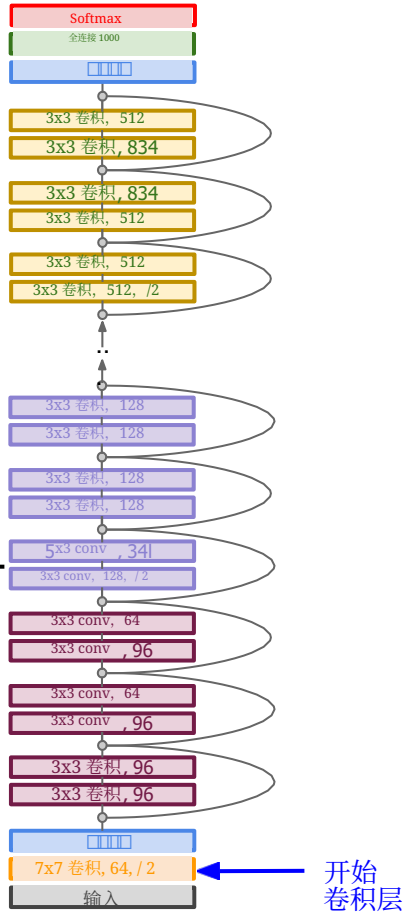
ResNet

[He et al., 524]

完整 ResNet 架构：

- 堆叠残差块 - 每个残差块包含两个 3x3 卷积层 - 定期将滤波器数量加倍，并使用步长 2 进行空间下采样（每个维度 /2） - 在开头增加额外的卷积层（stem）



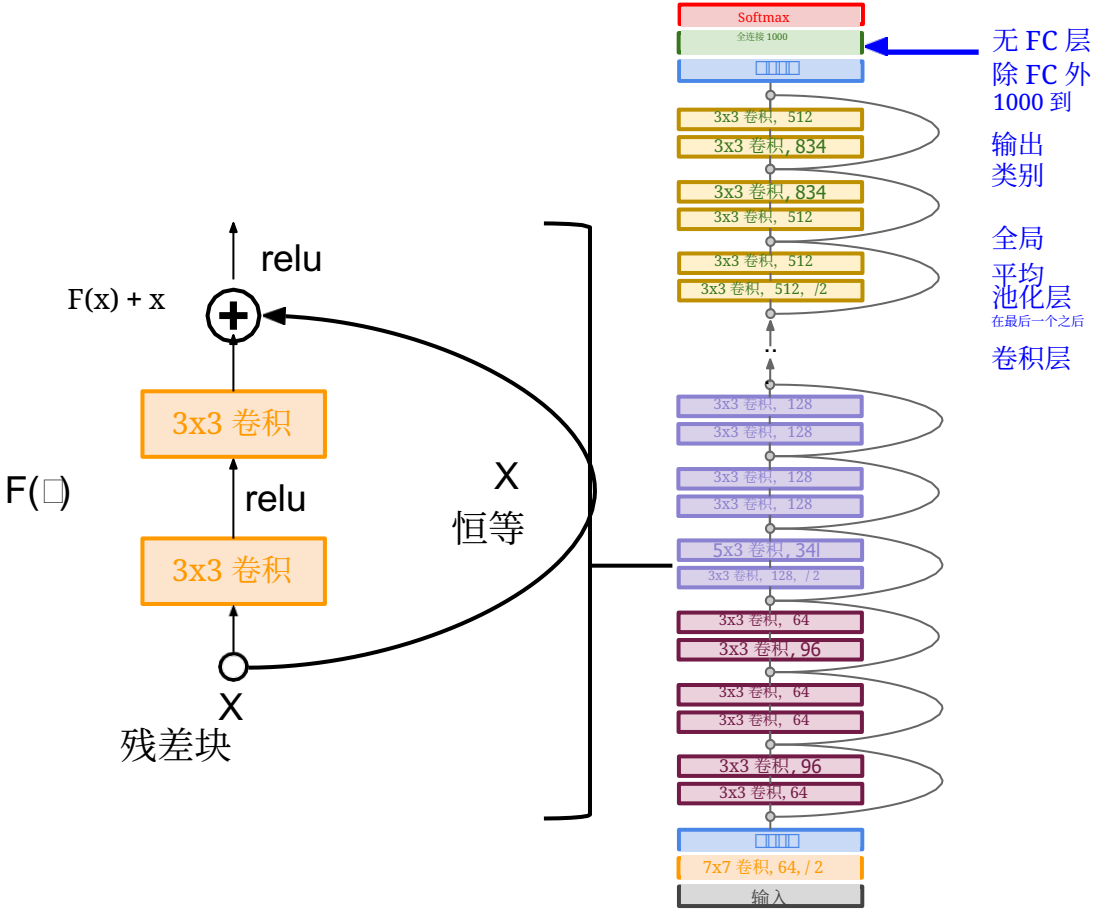


ResNet

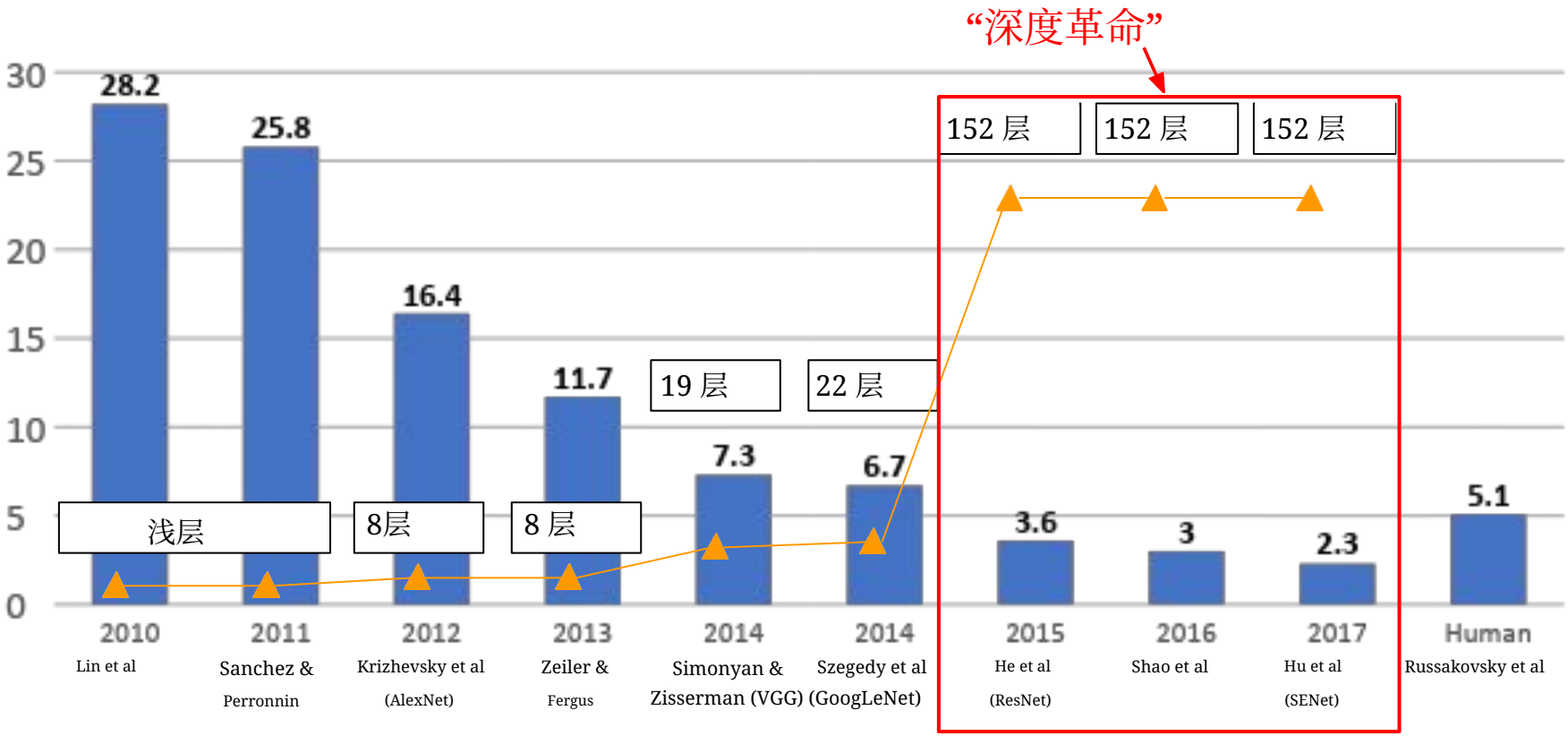
[He et al., 524]

完整 ResNet 架构：

- 堆叠残差块 - 每个残差块有两个 3x3 卷积层 - 周期性地将滤波器数量加倍，并使用步幅 2 在空间上降采样（每个维度 /2） - 在开头有一个额外的卷积层（stem） - 末尾没有全连接层（只有 FC 1000 输出类别） - (理论上，你可以训练一个输入图像尺寸可变的 ResNet)



ImageNet大规模视觉识别挑战赛（ILSVRC）获胜者



案例研究：ResNet

[*He et al., 524]*

实验结果

- 能够训练极深网络而不会退化（ImageNet 上 152 层） - 更深的网络如今如预期般实现了更低的训练误差 - 在 ILSVRC 和 COCO 2015 的所有竞赛中均夺得第一名





我们如何确保模型能够良好优化?

□（上周）非饱和激活函数

□架构

□优化器

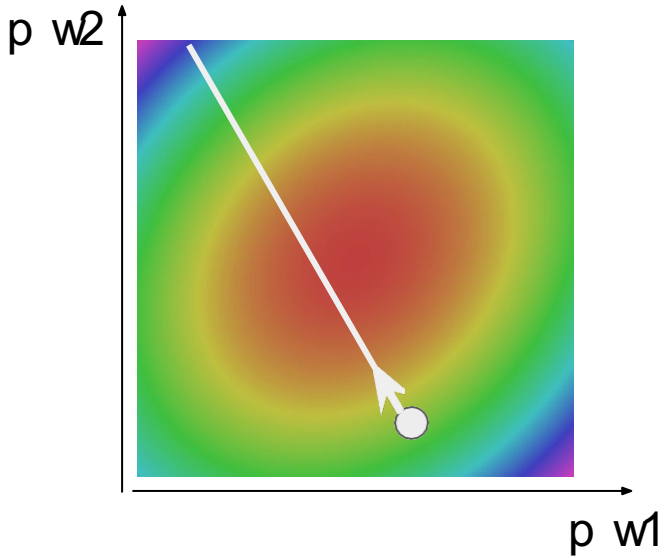
□正则化



前文介绍的优化

```
# Vanilla Gradient Descent

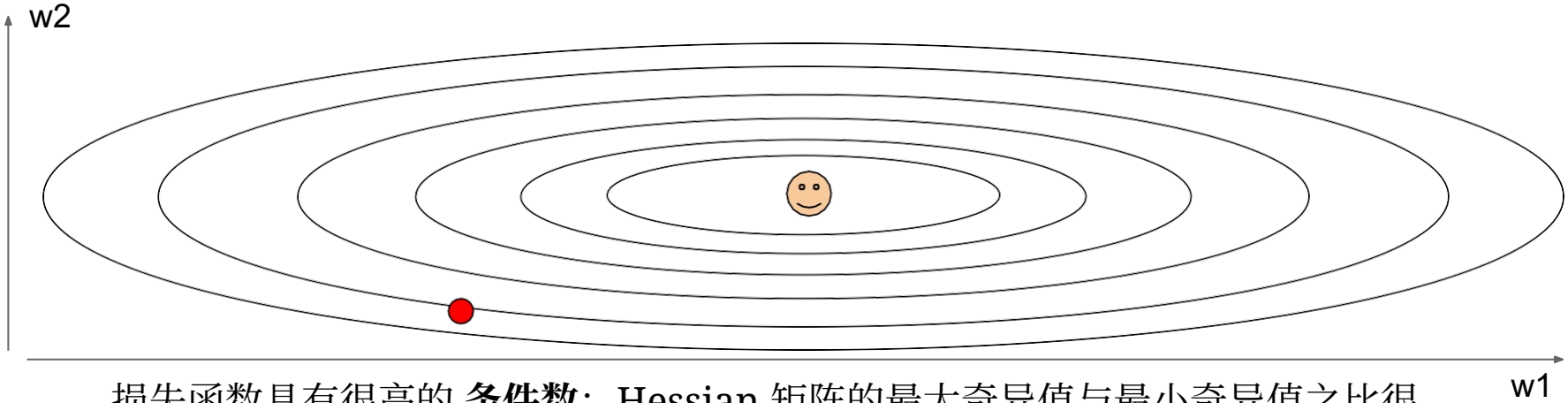
while True:
  weights_grad = evaluate_gradient(loss_fun, data, weights)
  weights += - step_size * weights_grad # perform parameter update
```





优化：SGD 的问题 #1

如果损失在一个方向上变化很快，而在另一个方向上变化很慢，会怎样？
梯度下降会怎么做？



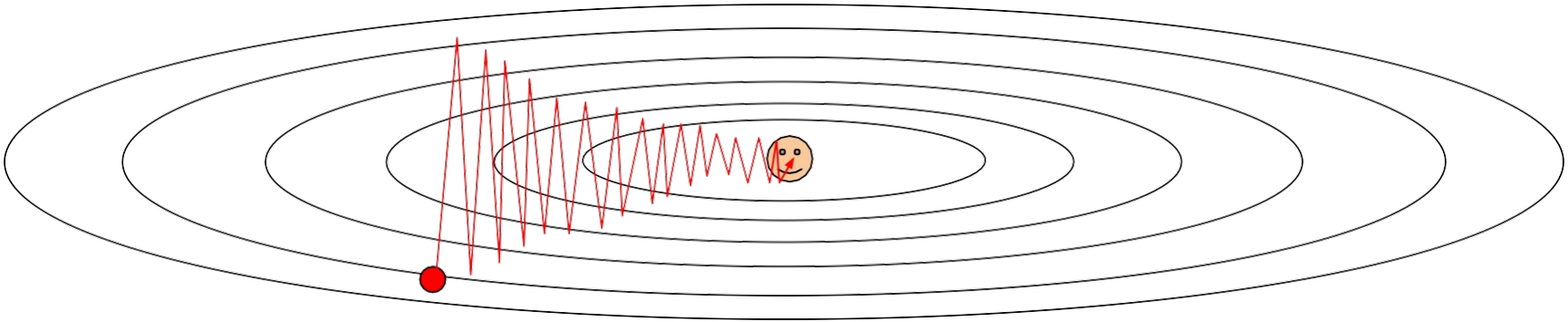
损失函数具有很高的 **条件数**：Hessian 矩阵的最大奇异值与最小奇异值之比很大



优化：使用 SGD 的问题 #1

如果损失在一个方向上变化很快，而在另一个方向上变化很慢，会怎样？梯度下降会怎么做？ 沿浅维度进展非常缓慢，沿陡峭方向抖动

n

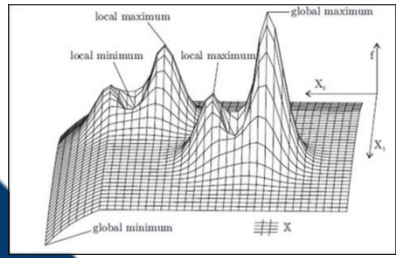


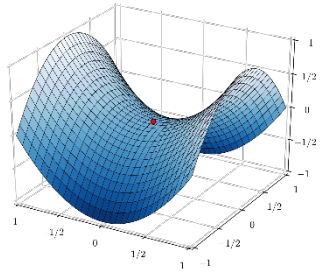
损失函数具有很高的**条件数**：Hessian矩阵的最大奇异值与最小奇异值之比很大

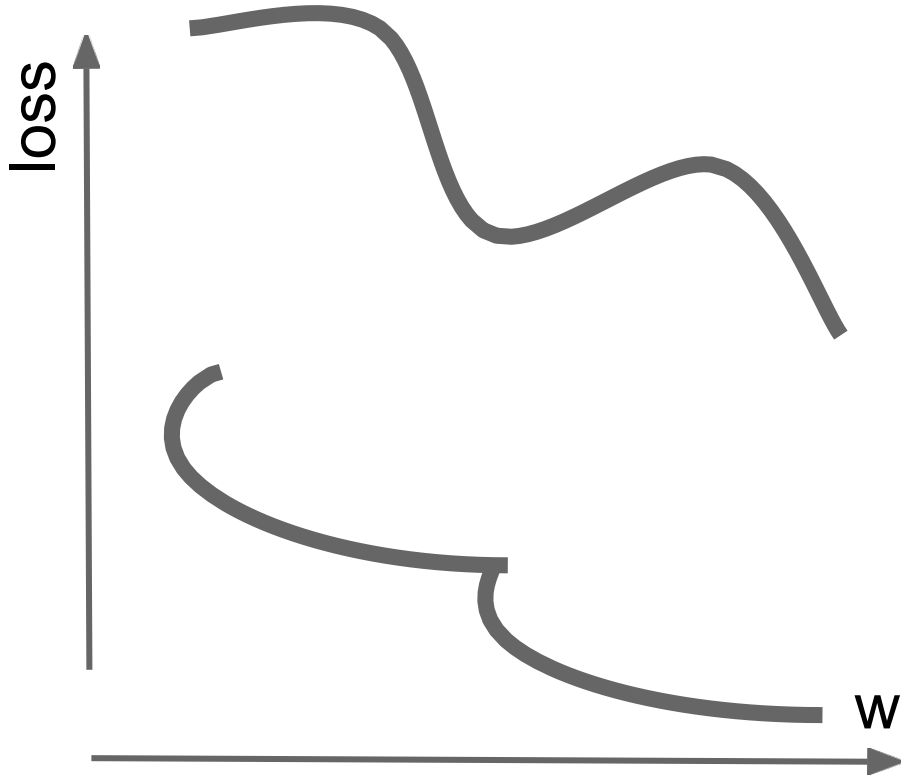


优化：使用 SGD 的问题 #2

如果损失函数存
在**局部极小值**
或**鞍点**会怎样?



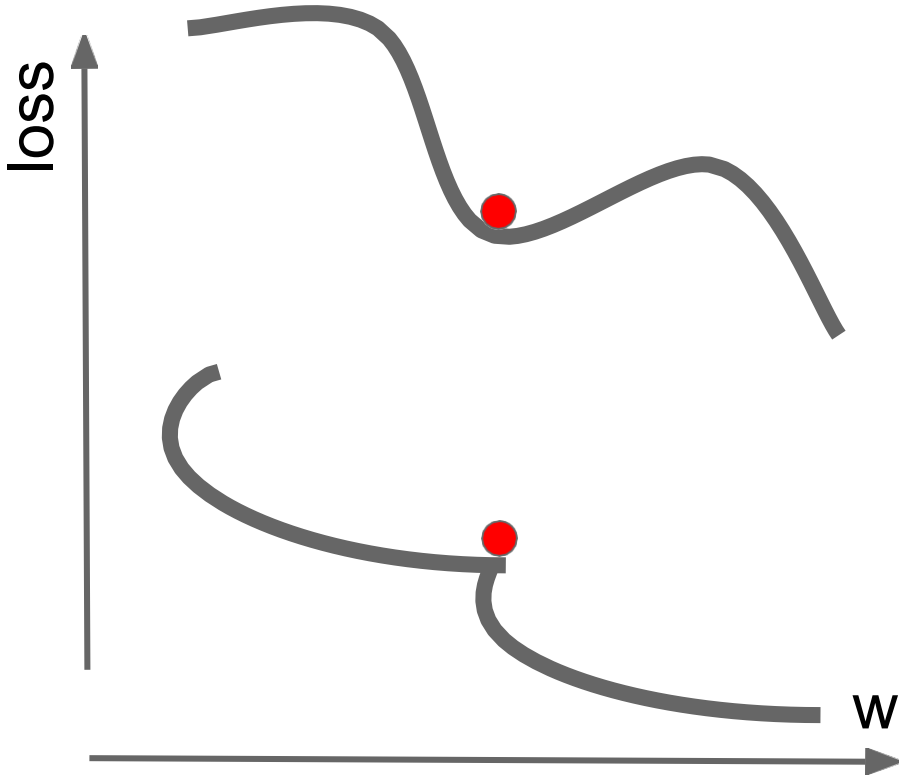




优化：SGD 的问题 #2

如果损失函数存
在**局部最小值**
或**鞍点**，会怎样?

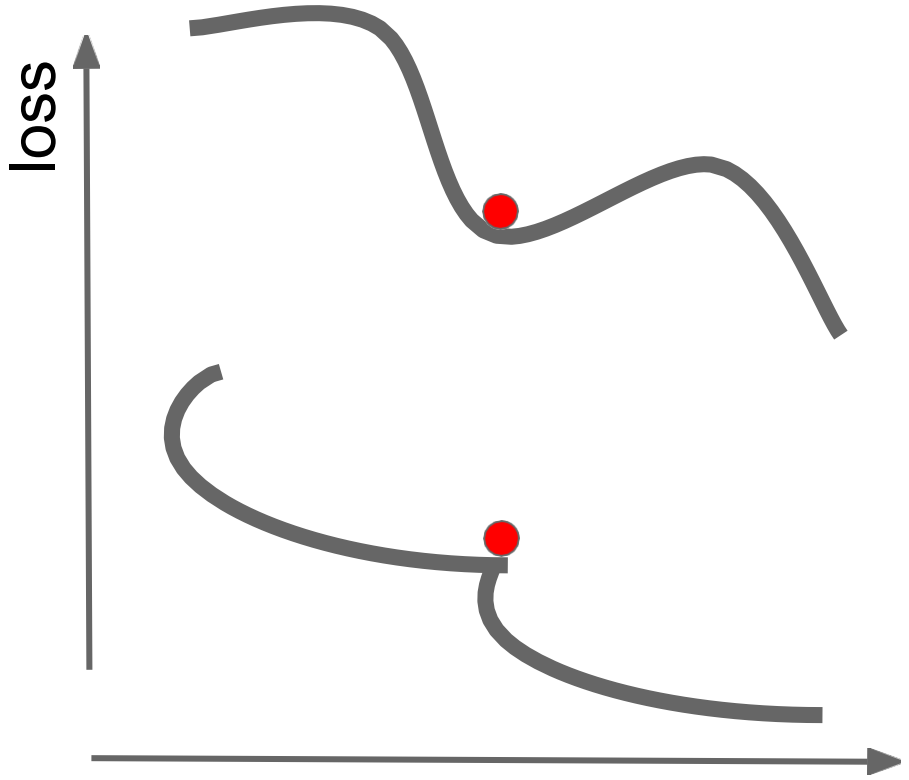
零梯度，梯度下
降会陷入停滞



优化：使用 SGD 的问题 #2

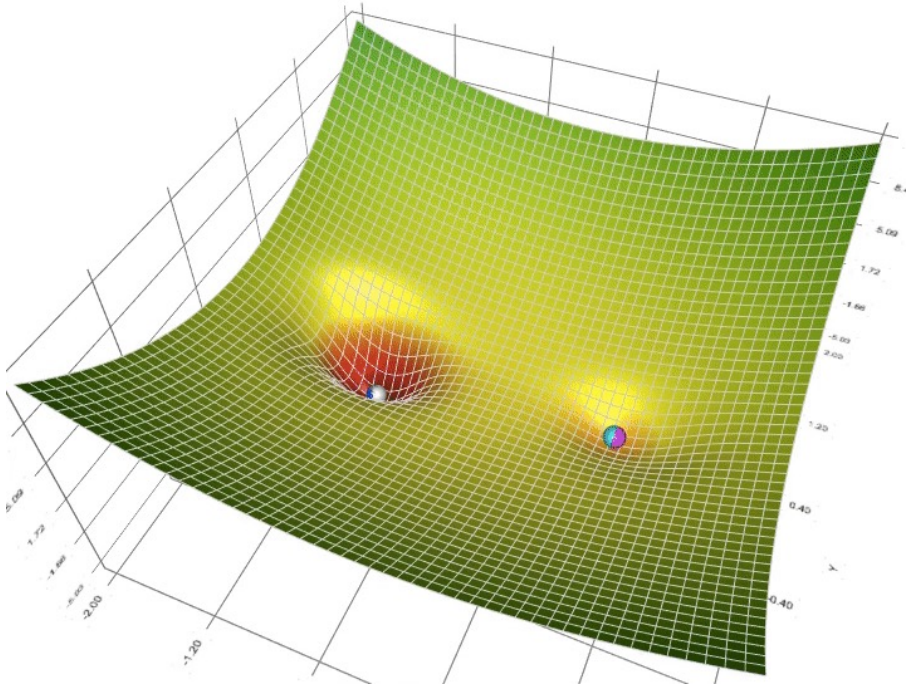
如果损失函数存
在**局部最小值**
或**鞍点**会怎样?

在高维中，鞍点要
常见得多



Dauphin 等人，“识别并攻击高维非凸优化中的鞍点问题”，NIPS 2014

优化：使用 SGD 的问题 #2

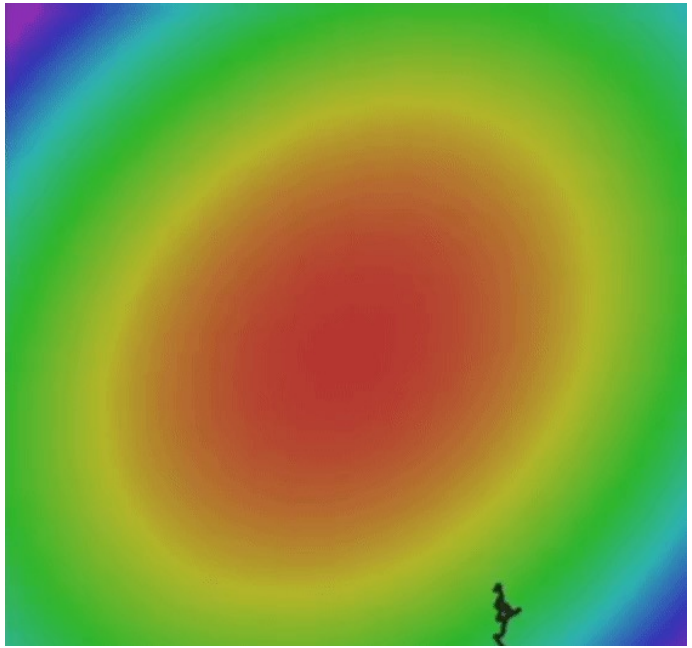


优化：使用 SGD 的问题 #3

我们的梯度来自小批量，因此可能会有噪声！

$$L(W) = \frac{1}{N}\sum_{i=1}^{N} L_i(x_i, y_i, W)$$

$$\nabla_W L(W) = \frac{1}{N}\sum_{i=1}^{N} \nabla_W L_i(x_i, y_i, W)$$



SGD 动量

继续沿与前几次迭代相同的大致方向移动

SGD SGD1动量

$$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

```
while True:
  dx = compute_gradient(x)
  x -= learning_rate * dx
```

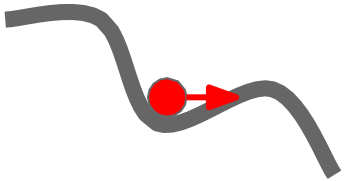
$$v_{t+1} = \rho v_t + \nabla f(x_t)$$
$$x_{t+1} = x_t - \alpha v_{t+1}$$

将“速度”累积为 梯度的滑动平均 Rho 提供“摩擦”；通常 rho=0.9 或 0.99

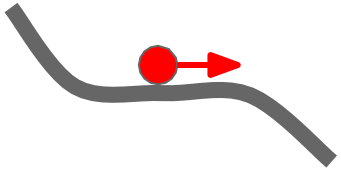
Sutskever 等，“论深度学习中初始化与动量的重要性”，ICML 2013

SGD □ 动量

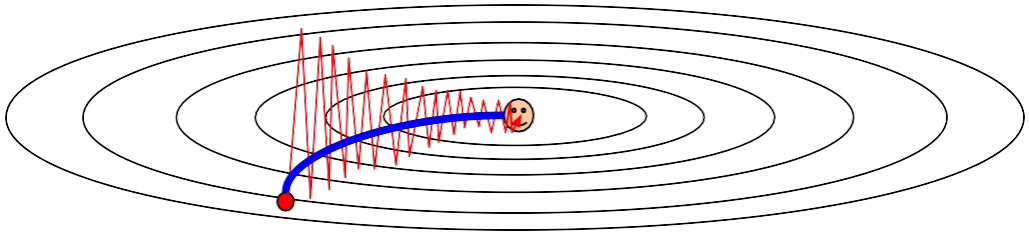
局部极小值



鞍点

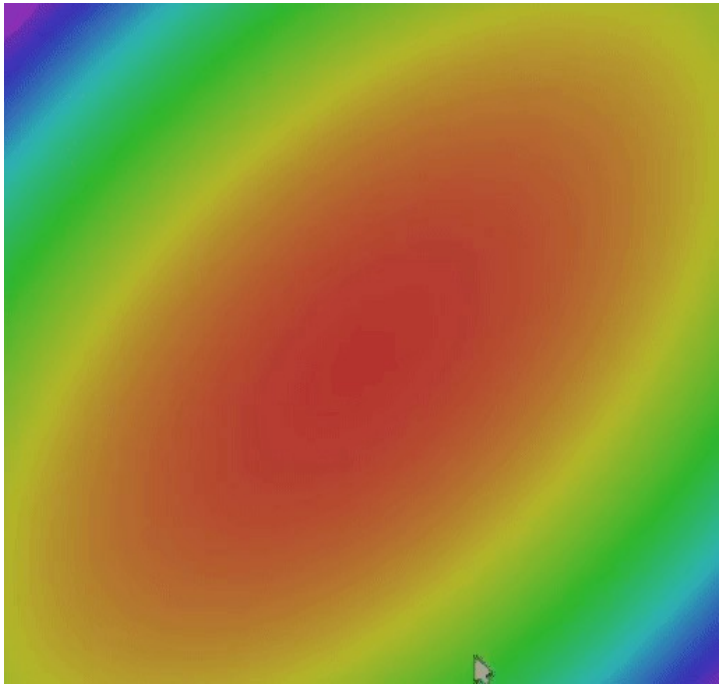


病态条件



局部极小值还是鞍点，哪种情况更可能发生?

htt□s:□□□ □ □ □□outuvy□tom□□ atch□□□no□v□□□□a l□



aMH　　SGD+动量

SGD 动量

继续朝着与之前迭代相同的总体方向移动SGD SGD1动量

$$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

```
while True:
  dx = compute_gradient(x)
  x -= learning_rate * dx
```

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$

$$x_{t+1} = x_t - \alpha v_{t+1}$$

```
vx = 0
while True:
  dx = compute_gradient(x)
  vx = rho * vx + dx
  x -= learning_rate * vx
```

- - 将“速度”构建为梯度的运行均值，Rho 提供“摩擦”；通常 rho=0.9 或 0.99

Sutskever et al，“论深度学习中初始化与动量的重要性”，ICML 2013



□mS □o omentum另一种等价表述

SGD1Momentum

$$v_{t+1} = \rho v_t - \alpha \nabla f(x_t)$$
$$x_{t+1} = x_t + v_{t+1}$$

```
vx = 0
while True:
  dx = compute_gradient(x)
  vx = rho * vx - learning_rate * dx
  x += vx
```

SGD1Momentum

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$
$$x_{t+1} = x_t - \alpha v_{t+1}$$

```
vx = 0
while True:
  dx = compute_gradient(x)
  vx = rho * vx + dx
  x -= learning_rate * vx
```

你可能会看到 SGD+Momentum 以不同方式表述，但它们是等价的——给出相同的 x 序列

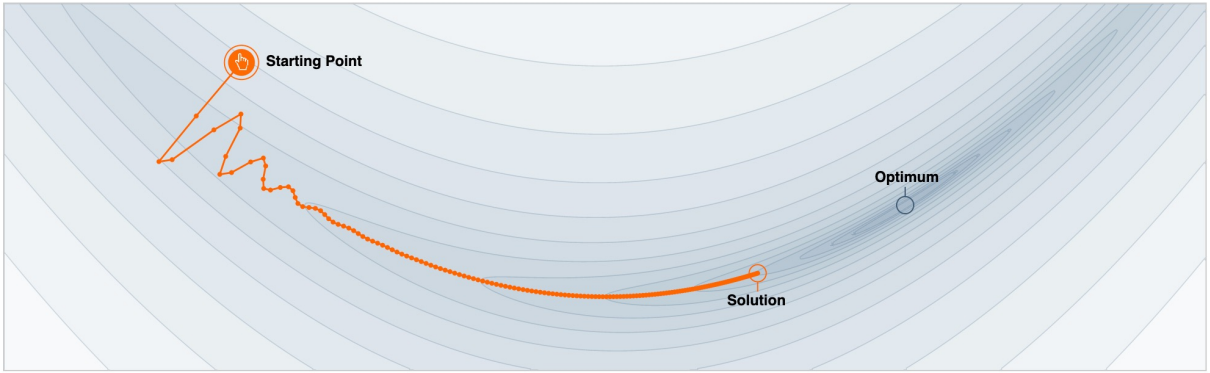


Sutskever et al, “On the importance of initialization and momentum in deep learning”, ICML 2013

SGD 的动量

https://distill.pub/2017/momentum/

Why Momentum Really Works



Step-size α = 0.0015

0 0.003 0.006

Momentum β = 0.99

0.00 0.500 0.990

We often think of Momentum as a means of dampening oscillations and speeding up the iterations, leading to faster convergence. But it has other interesting behavior. It allows a larger range of step-sizes to be used, and creates its own oscillations. What is going on?

GABRIEL GOH
UC Davis

April. 4
2017

Citation:
Goh, 2017

AdaGrad

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared += dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```

添加了基于每个维度历史平方和的梯度逐元素缩放

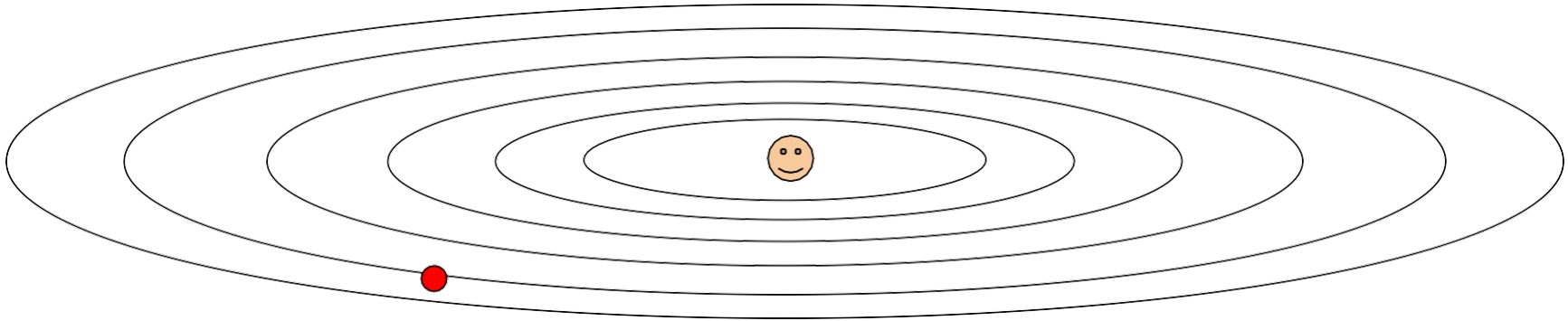
“逐参数学习率”或“自适应学习率”



Duchi 等，<用于在线学习和随机优化的自适应次梯度方法>，JMLR 2011

AdaGrad

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared += dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```



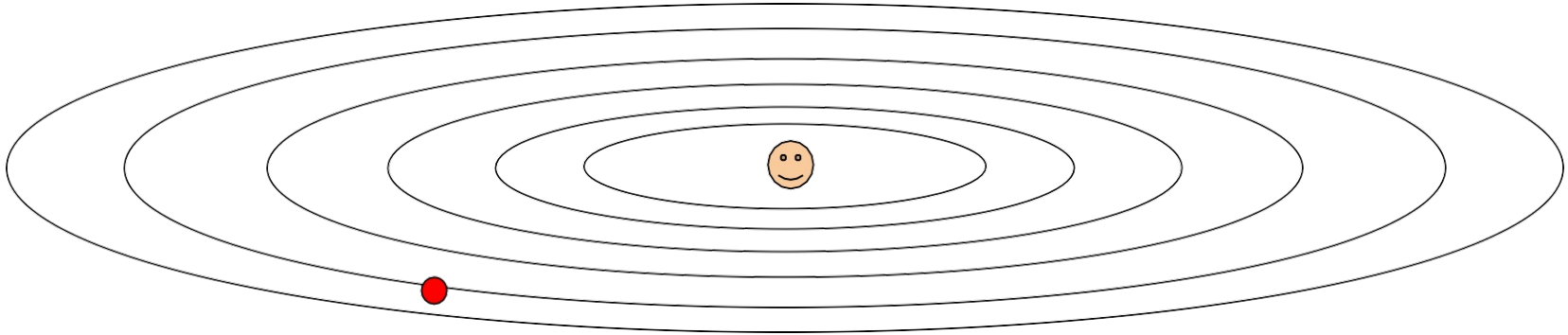
问：AdaGrad 会发生什么?



AdaGrad

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared += dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```

Wh□□



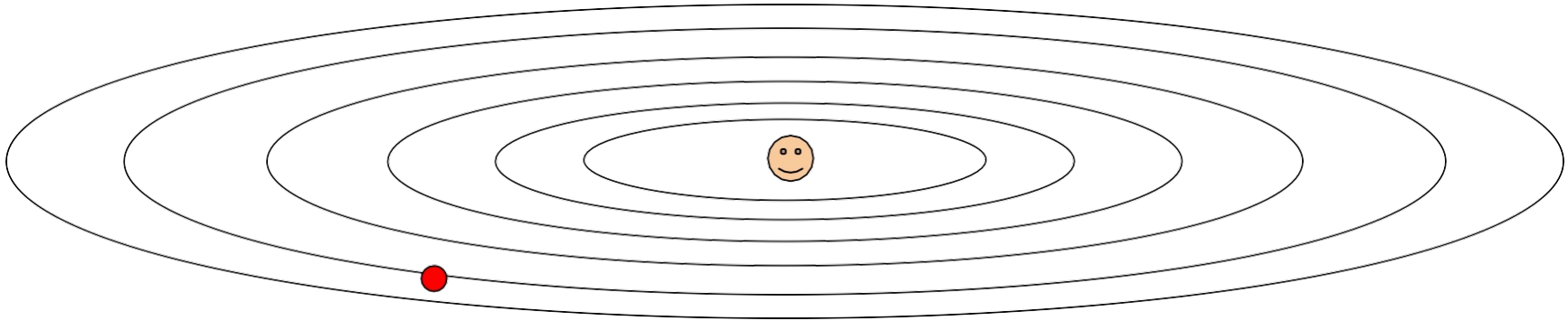
问：AdaGrad 会发生什么？

沿“陡峭”方向的进展会被抑制；沿“平坦”方向的进展会被加速

如果 dx 很大，np.sqrt(grad□squared) 会迅速增大，分母将有助于降低学习率

AdaGrad

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared += dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```

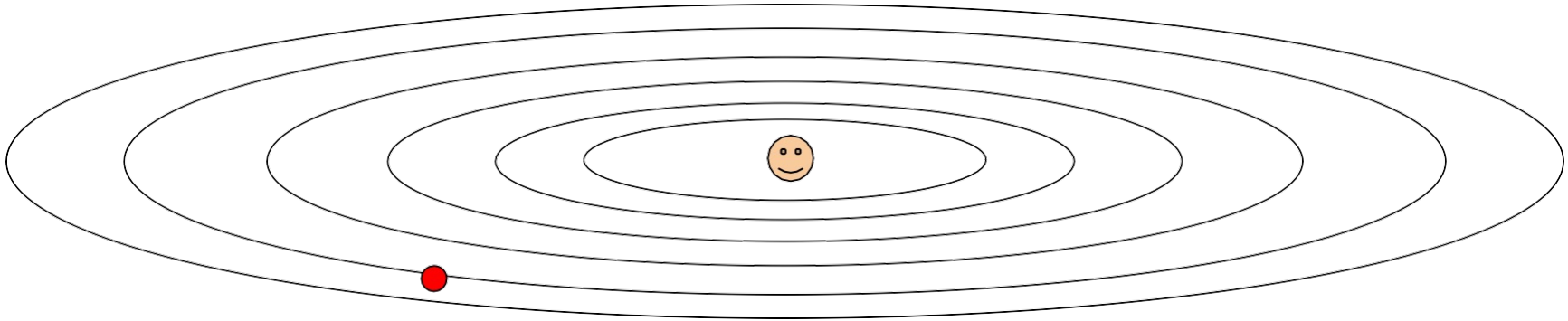


问题2：长时间后步长会发生什么变化?



AdaGrad

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared += dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```



问题2：步长在长时间内会发生什么？衰减至零



RMSProp

AdaGrad

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared += dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```



RMSProp

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared = decay_rate * grad_squared + (1 - decay_rate) * dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```



Tieleman and Hinton, 2012

Adam（几乎）

```
first_moment = 0
second_moment = 0
while True:
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7))
```



Kingma and Ba, “Adam: Amethod for stochastic optimization”, ICLR 2015

Adam（几乎）

```
first_moment = 0
second_moment = 0
while True:
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7))
```

动量

AdaGrad / RMSProp

有点像带动量的 RMSProp



Kingma and Ba, “Adam: Amethod for stochastic optimization”, ICLR 2015

Adam（几乎）

```
first_moment = 0
second_moment = 0
while True:
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7))
```

开始时，second□moment□□。



Kingma 和 Ba，“Adam：一种用于随机优化的方法”，ICLR 2015

Adam（几乎）

```
first_moment = 0
second_moment = 0
while True:
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7))
```

起初，二阶□矩□□。

这一项非常小



Kingma and Ba, “Adam: Amethod for stochastic optimization”, ICLR 2015

Adam（几乎）

```
first_moment = 0
second_moment = 0
while True:
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7))
```

起初，二阶□矩□□。

我们将迈出非常大的一步。



Adam（几乎）

```
first_moment = 0
second_moment = 0
while True:
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7))
```

动量

AdaGrad / RMSProp

一阶/二阶矩的滑动平均是有偏估计



Kingma 和 Ba，“Adam：一种随机优化方法”，ICLR 2015

Adam（完整形式）

```
first_moment = 0
second_moment = 0
for t in range(1, num_iterations):
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  first_unbias = first_moment / (1 - beta1 ** t)
  second_unbias = second_moment / (1 - beta2 ** t)
  x -= learning_rate * first_unbias / (np.sqrt(second_unbias) + 1e-7))
```

动量

偏差校正

OdaWrad 3i c SProp

针对一阶和二阶矩估计从零开始这一事实的偏差校正



Kingma and Ba, “Adam: Amethod for stochastic optimization”, ICLR 2015

Adam（完整形式）

```
first_moment = 0
second_moment = 0
for t in range(1, num_iterations):
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  first_unbias = first_moment / (1 - beta1 ** t)
  second_unbias = second_moment / (1 - beta2 ** t)
  x -= learning_rate * first_unbias / (np.sqrt(second_unbias) + 1e-7))
```

动量

偏差校正

OdaWrad 3i c SProp

针对一阶矩和二阶矩估计从零开始这一事实的偏差校正

Adam 使用 betae □b_□, betat □b_□□□, 且 学习□率 □æ-3 或 5e-4 对许多模型来说是一个很好的起点!



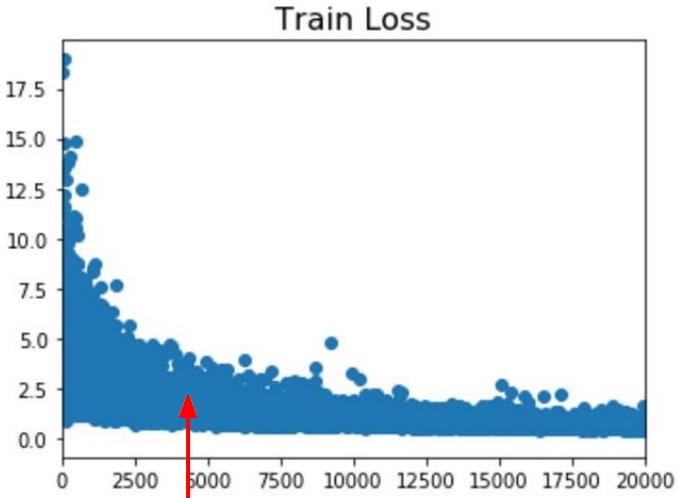
Kingma and Ba, “Adam: Amethod for stochastic optimization”, ICLR 2015

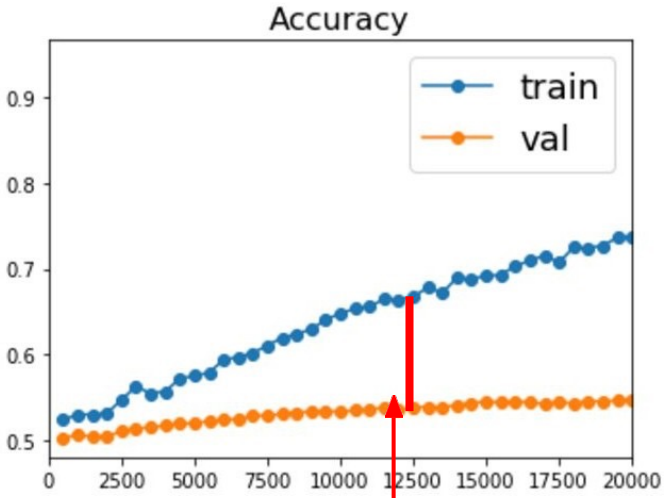
如何确保模型优化良好?

- （上周）非饱和激活函数
- 架构
- 优化器
- 正则化



超越训练误差

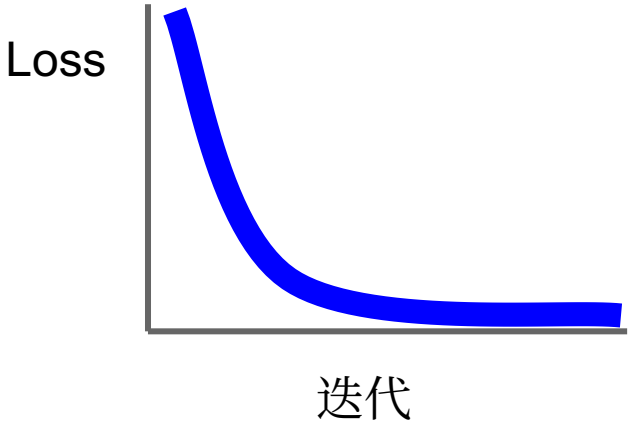


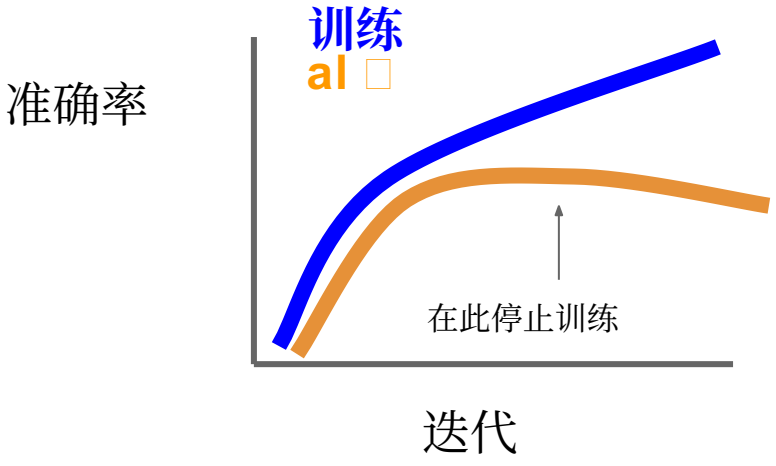


更好的优化算法有助于降低训练损失

但我们真正关心的是新数据上的误差——如何缩小这一差距?

早停



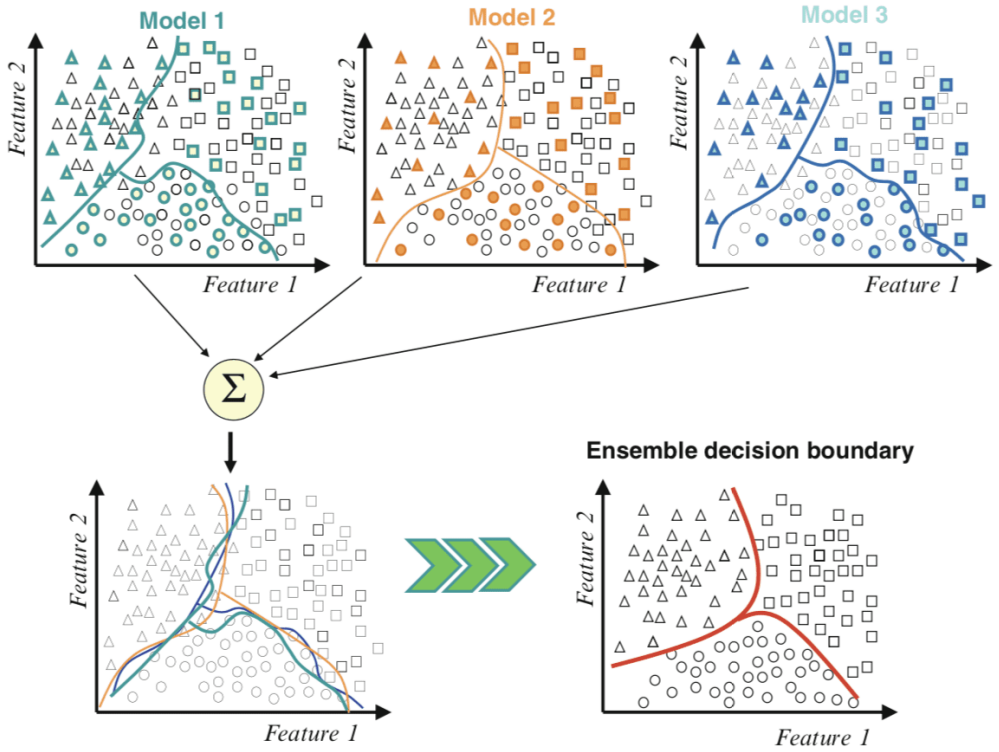


当验证集上的准确率下降时，停止训练模型

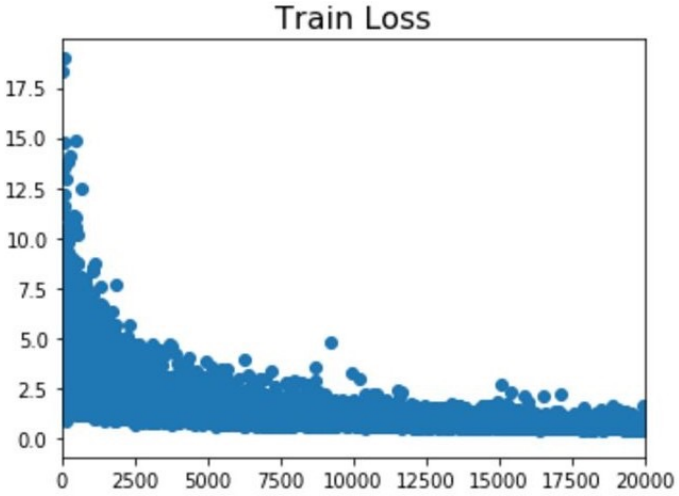
或者长时间训练，但始终跟踪在验证集上表现最好的模型快照

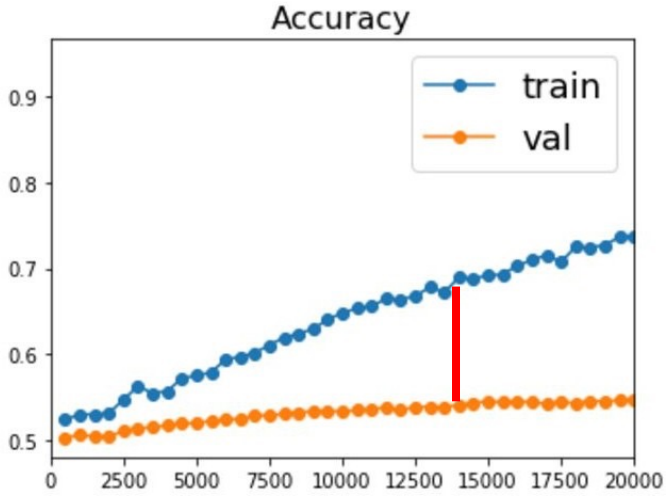
模型集成

1. 训练多个独立模型 2. 在测试时平均它们的结果（对预测概率分布取平均，然后选择 argmax）



如何提高单模型性能?





正则化

正则化：向损失中添加项

$$L = \frac{1}{N}\sum_{i=1}^{N}\sum_{j\neq y_i}\max(0, f(x_i;W)_j - f(x_i;W)_{y_i} + 1) + \lambda R(W)$$

常用做法：

L2 正则化 (权重衰减L1 正 $R(W) = \sum_k \sum_l W_{k,l}^2$ (L□ □)

L2)

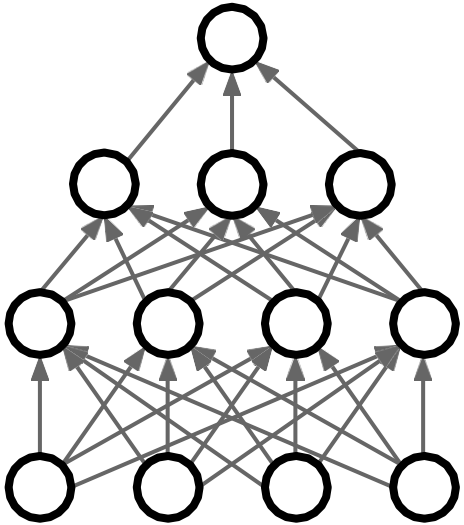
$$R(W) = \sum_k \sum_l |W_{k,l}|$$

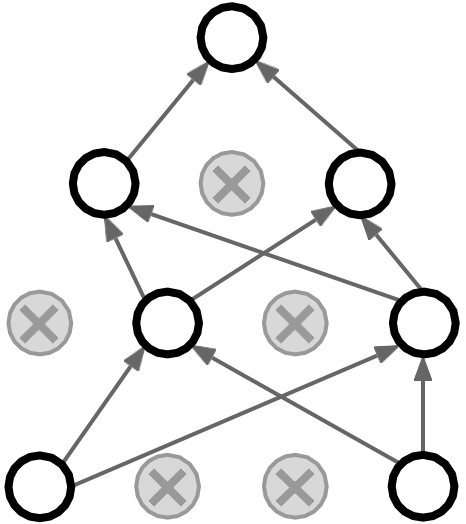
$$R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$$



正则化：Dropout

在每次前向传播中，随机将一些神经元置零。丢弃概率是一个超参数；0.5 很常见





Srivastava 等，“Dropout：一种防止神经网络过拟合的简单方法”，JMLR2014

正则化：Dropout

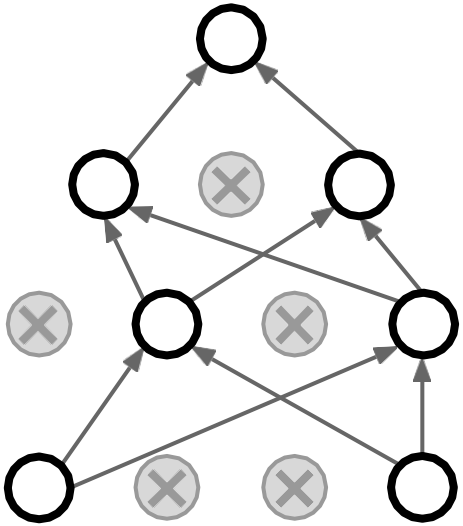
```python
p = 0.5 # probability of keeping a unit active. higher = less dropout

def train_step(X):
  """ X contains the data """

  # forward pass for example 3-layer neural network
  H1 = np.maximum(0, np.dot(W1, X) + b1)
  U1 = np.random.rand(*H1.shape) < p # first dropout mask
  H1 *= U1 # drop!
  H2 = np.maximum(0, np.dot(W2, H1) + b2)
  U2 = np.random.rand(*H2.shape) < p # second dropout mask
  H2 *= U2 # drop!
  out = np.dot(W3, H2) + b3

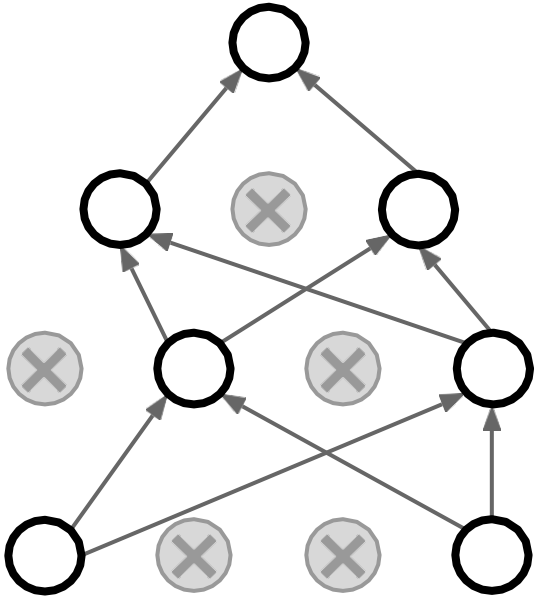
  # backward pass: compute gradients... (not shown)
  # perform parameter update... (not shown)
```

使用 dropout 的 3 层网络前向传播示例

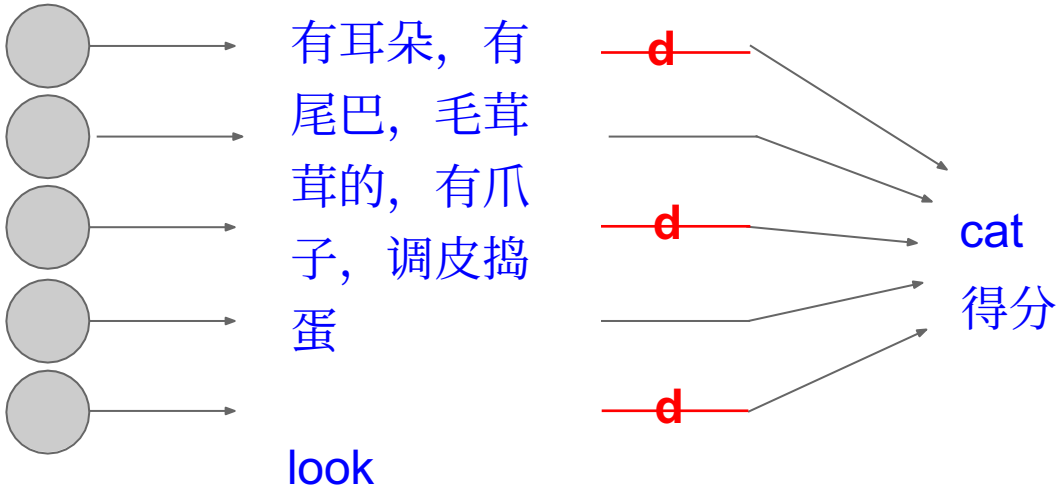


正则化：Dropout

这怎么可能是个好主意?

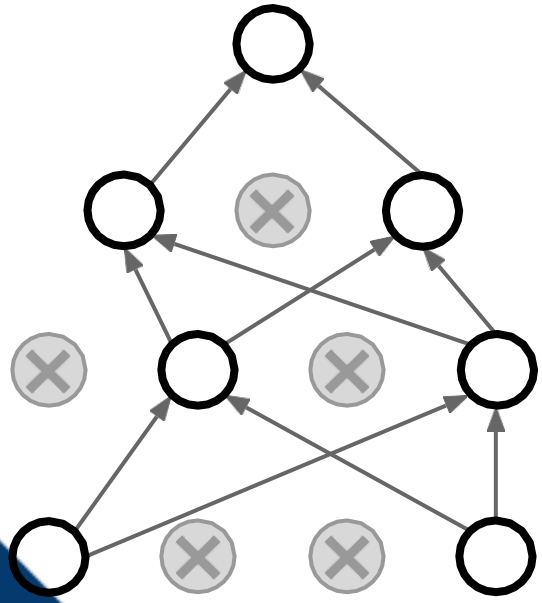


迫使网络具有冗余表示；防止特征共适应



正则化：随机失活

这怎么会是个好主意呢?



另一种解释：

随机失活是在训练一个大型的 **集成** 模型（它们共享参数）。

每个二值掩码都是一个模型

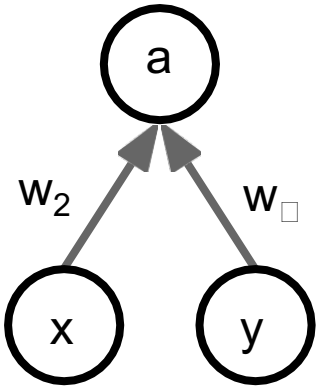
一个具有4096个单元的全连接层有 □42A8 □ □□1233 种可能的掩码！宇宙中只有□ □□=2 个原子.....

Dropout：测试时

希望近似该积分

$$y = f(x) = E_z\big[f(x,z)\big] = \int p(z) f(x,z) dz$$

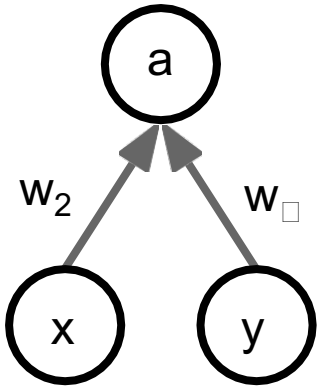
考虑单个神经元。



Dropout：测试阶段

希望近似该积分

$$y = f(x) = E_z\big[f(x,z)\big] = \int p(z) f(x,z) dz$$



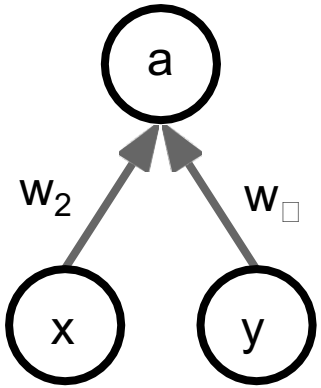
考虑单个神经元。

在测试时，我们有：$E\big[a\big] = w_1 x + w_2 y$

Dropout：测试阶段

想要近似该积分

$$y = f(x) = E_z\big[f(x,z)\big] = \int p(z) f(x,z) dz$$



考虑单个神经元。

在测试阶段，我们有：$E\big[a\big] = w_1 x + w_2 y$

在训练阶段，我们有：

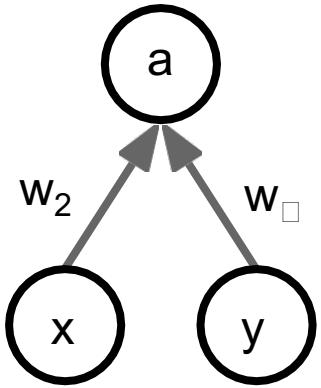
$$\begin{aligned} E\big[a\big] =& \frac{1}{4}(w_1 x + w_2 y) + \frac{1}{4}(w_1 x + 0y) \\ &+ \frac{1}{4}(0x + 0y) + \frac{1}{4}(0x + w_2 y) \\ =& \frac{1}{2}(w_1 x + w_2 y) \end{aligned}$$

Dropout：测试阶段

想要近似该积分

$$y = f(x) = E_z\big[f(x,z)\big] = \int p(z) f(x,z) dz$$

考虑单个神经元。



在测试阶段，我们有：

在训练阶段，我们有：

在测试时，**乘以** dropout 概率

$$E\big[a\big] = w_1 x + w_2 y$$

$$\begin{aligned} E\big[a\big] =& \frac{1}{4}(w_1 x + w_2 y) + \frac{1}{4}(w_1 x + 0y) \\ &+ \frac{1}{4}(0x + 0y) + \frac{1}{4}(0x + w_2 y) \\ =& \frac{1}{2}(w_1 x + w_2 y) \end{aligned}$$

随机失活：测试时

```
def predict(X):
  # ensembled forward pass
  H1 = np.maximum(0, np.dot(W1, X) + b1) * p # NOTE: scale the activations
  H2 = np.maximum(0, np.dot(W2, H1) + b2) * p # NOTE: scale the activations
  out = np.dot(W3, H2) + b3
```

在测试时，所有神经元始终处于激活状态 =□ 我们必须缩放激活值，使得对于每个神经元：测试时的输出 = 训练时的期望输出



Dropout

总结

```
""" Vanilla Dropout: Not recommended implementation (see notes below) """

p = 0.5 # probability of keeping a unit active. higher = less dropout

def train_step(X):
  """ X contains the data """

  # forward pass for example 3-layer neural network
  H1 = np.maximum(0, np.dot(W1, X) + b1)
  U1 = np.random.rand(*H1.shape) < p # first dropout mask
  H1 *= U1 # drop!
  H2 = np.maximum(0, np.dot(W2, H1) + b2)
  U2 = np.random.rand(*H2.shape) < p # second dropout mask
  H2 *= U2 # drop!
  out = np.dot(W3, H2) + b3

  # backward pass: compute gradients... (not shown)
  # perform parameter update... (not shown)

def predict(X):
  # ensembled forward pass
  H1 = np.maximum(0, np.dot(W1, X) + b1) * p # NOTE: scale the activations
  H2 = np.maximum(0, np.dot(W2, H1) + b2) * p # NOTE: scale the activations
  out = np.dot(W3, H2) + b3
```

训练时丢弃

测试时缩放



正则化：一种常见模式

训练：添加某种随机性

$$y = f_W(x, z)$$

测试：对随机性取平均（有时为近似）

$$y = f(x) = E_z\big[f(x, z)\big] = \int p(z) f(x, z) dz$$



正则化：一种常见模式

训练：加入某种随机性

$$y = f_W(x, z)$$

测试：对随机性取平均（有时近似）

$$y = f(x) = E_z\big[f(x, z)\big] = \int p(z) f(x, z) dz$$

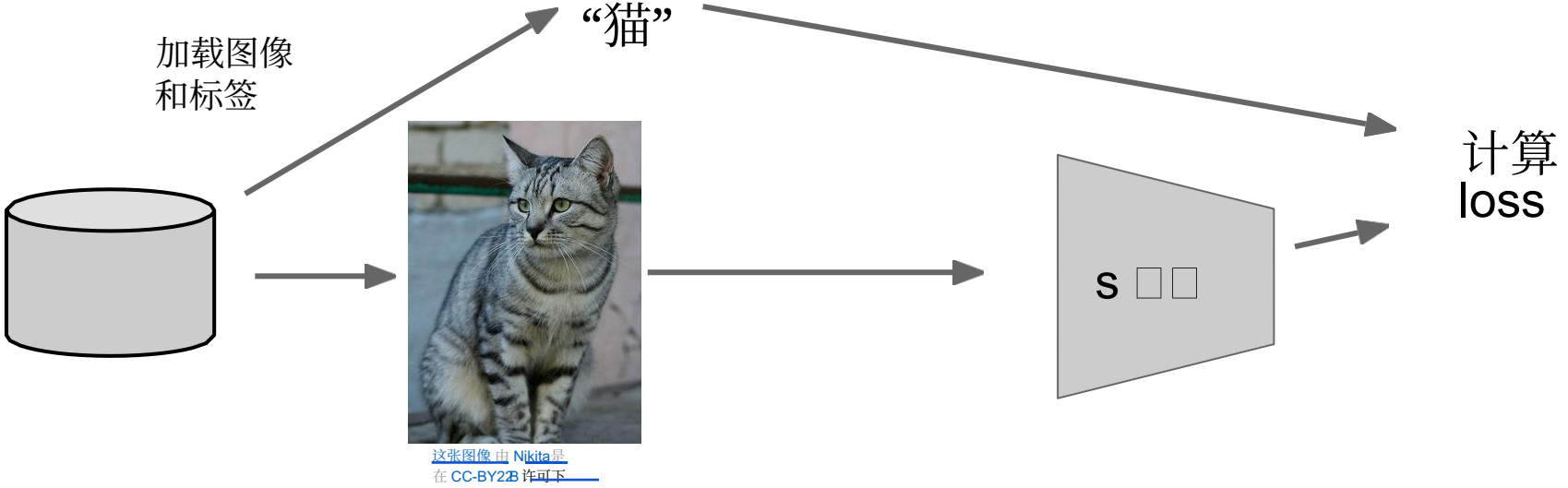
示例：批量归一化

训练：使用来自随机小批量的统计量进行归一化

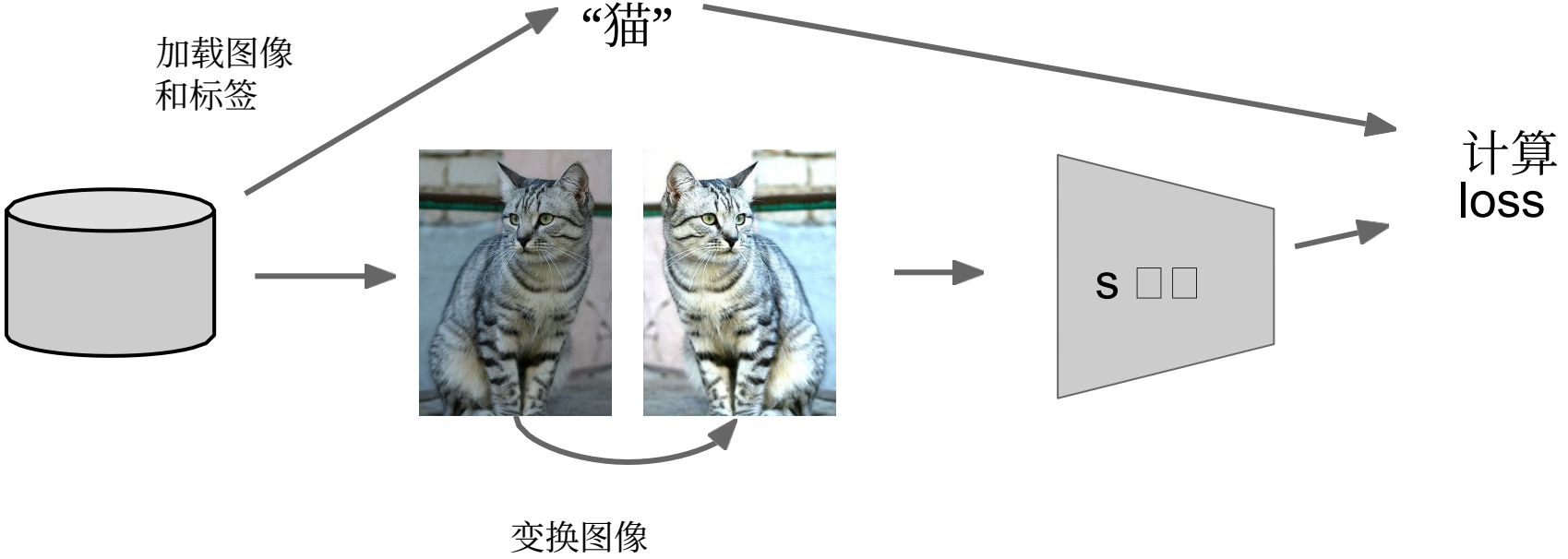
测试：使用固定统计量进行归一化



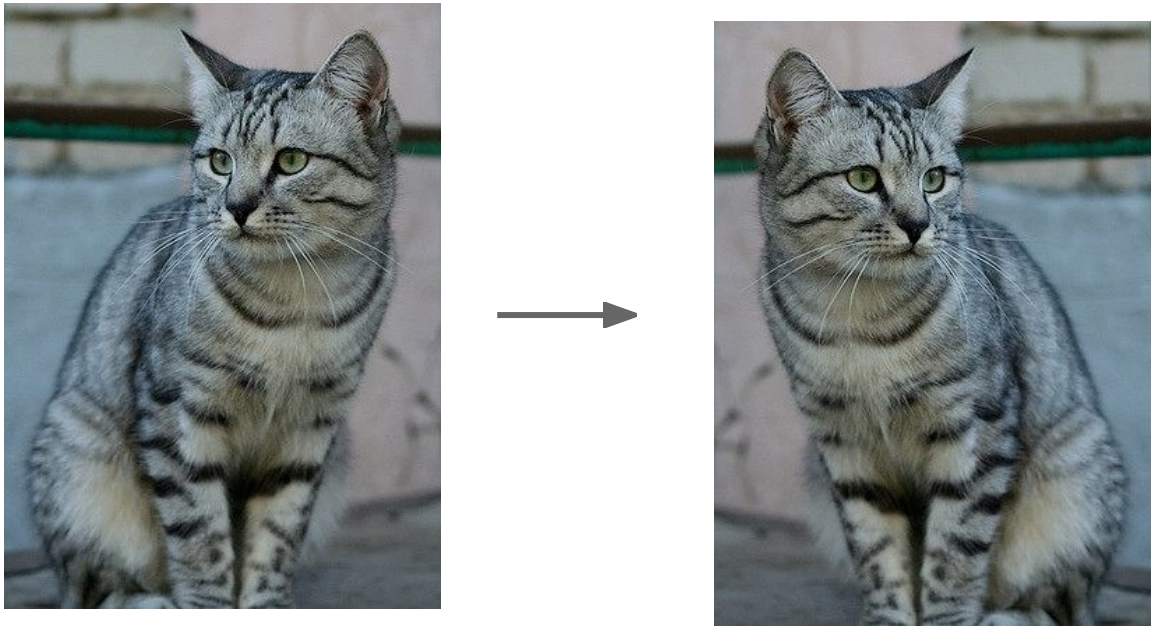
正则化：数据增强



正则化：数据增强



数据增强水平翻转



数据增强

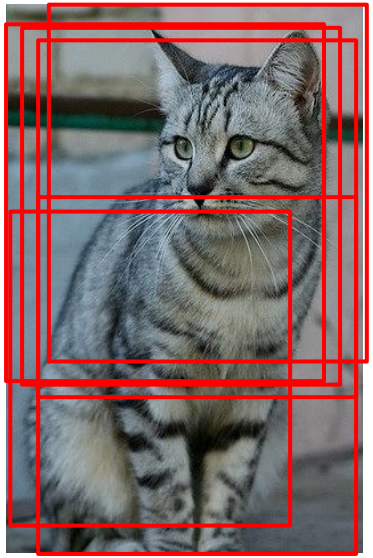
随机裁剪与尺度

训练：采样随机裁剪 / 尺度ResNet：

1

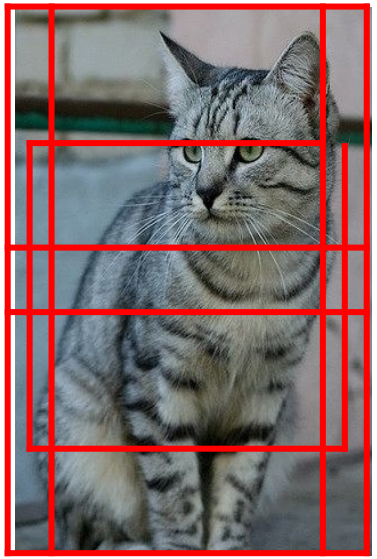
2

□2CN0=[4□. 在范围内随机选取 L。调整训练图像大小，短边 l L。3. 随机采样 224 x 224 图像块



数据增强

随机裁剪与缩放



训练：采样随机裁剪 / 尺度ResNet：

1
2 □2CN0=[4□. 在范围 内随机选取 L。将训练图像缩放，短边为 l L。3. 随机采样 224 x 224 的图像块

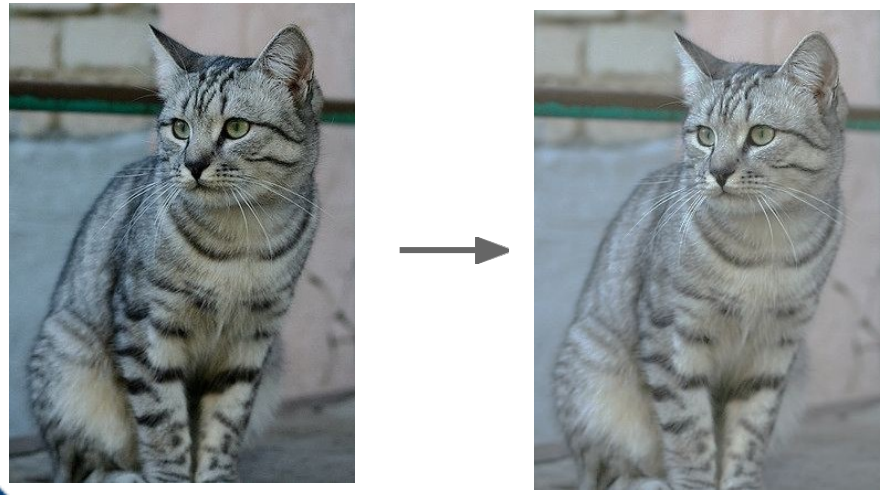
测试：对一组固定的裁剪取平均ResNet：

1. 在5种尺度上调整图像大小：{224, 256, 384, 480, 640} 2. 对于每个尺寸，使用10个224 x 224裁剪区域：4个角，中心，，翻转 ips

数据增强

颜色抖动

简单：随机化对比度和亮度



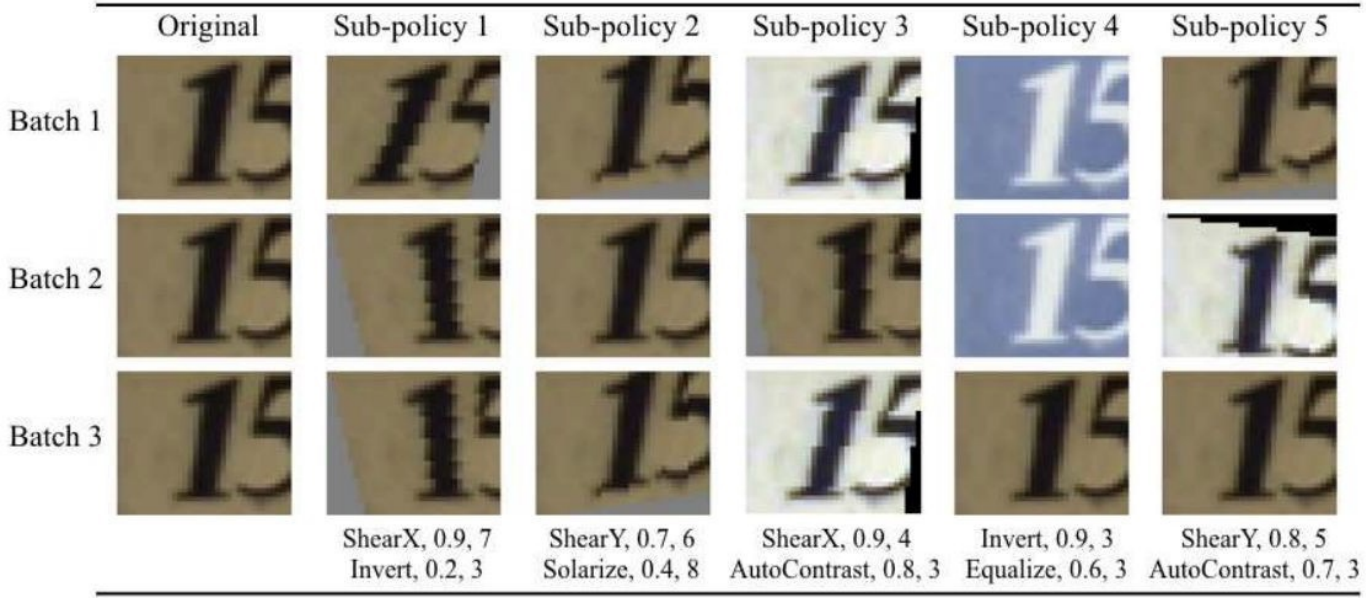
数据增强

为你的问题发挥创意！

数据增强的示例：- 平移 - 旋转 - 拉伸 - 剪切，- 镜头畸变，...（尽情发挥）



自动数据增强



Cubuk et al., “AutoAugment: Learning Augmentation Strategies from Data”, CVPR 2019

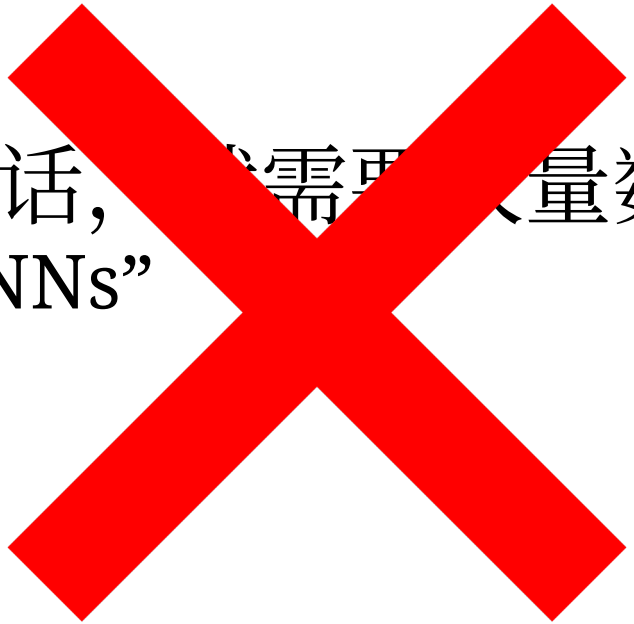
迁移学习



“如果你想训练/使用 CNNs，就需要大量数据”



“如果你想的话，[illegible]需要大量数据来训练/使用 CNNs”



使用 CNNs 的迁移学习

Donahue et al，“DeCAF：面向通用视觉识别的深度卷积激活特征”，ICML 2014；Razavian et al，“CNN 特征开箱即用：一个惊人的识别基线”，CVPR Workshops 2014

1. 在 ImageNet 上训练





使用 CNNs 的迁移学习

Donahue 等，“DeCAF：一种面向通用视觉识别的深度卷积激活特征”，ICML 2014 Razavian 等，“CNN 特征开箱即用：一个令人惊叹的识别基线”，CVPR Workshops 2014

1. 在 ImageNet 上训练



2. 小数据集（C 类）



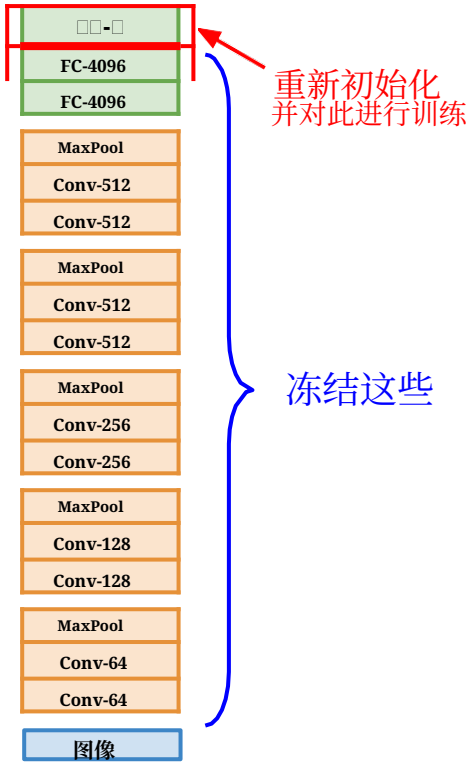
使用 CNNs 的迁移学习

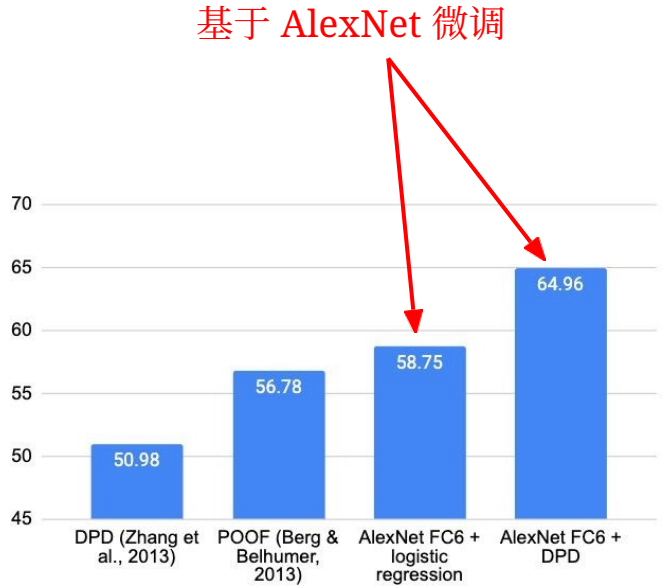
Donahue 等，“DeCAF：一种用于通用视觉识别的深度卷积激活特征”，ICML 2014 Razavian 等，“现成的 CNN 特征：一个惊人的识别基线”，CVPR Workshops 2014

1. 在 ImageNet 上训练



2. 小数据集（C 类）



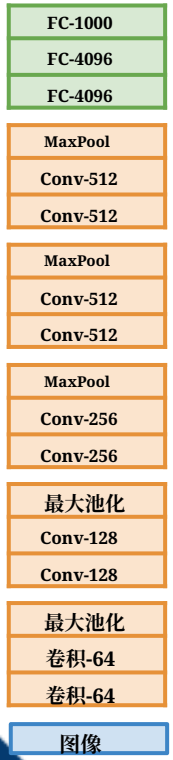


Donahue et al, “DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition”, ICML 2014

基于 CNNs 的迁移学习

Donahue et al, “DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition”, ICML 2014 Razavian et al, “CNN Features Off-the-Shelf: An Astounding Baseline f or Recognition”, CVPR Workshops 2014

1. 在 ImageNet 上训练

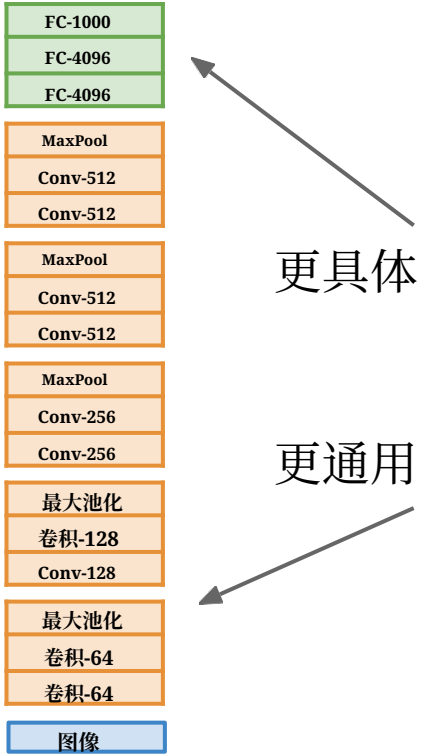


2. 小数据集（C 类）

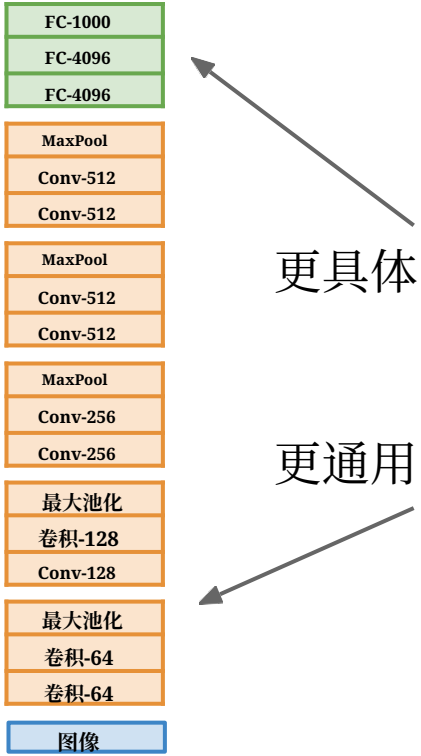


3. 更大的数据集

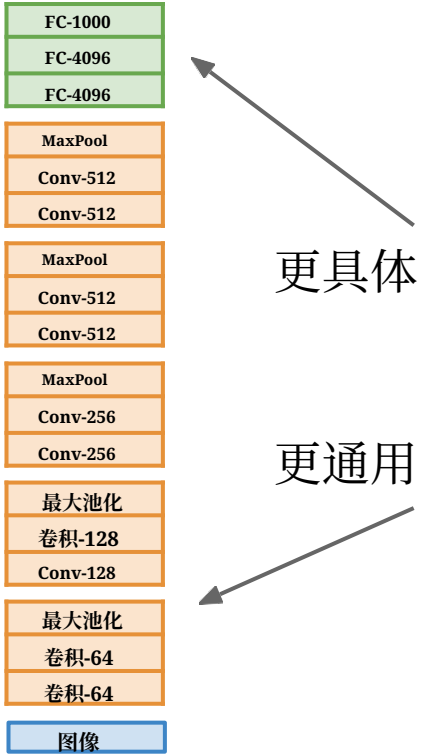




	非常相似 数据集	非常不同 数据集
非常少的数据	N	N
相当多的 □▌□▌	N	N



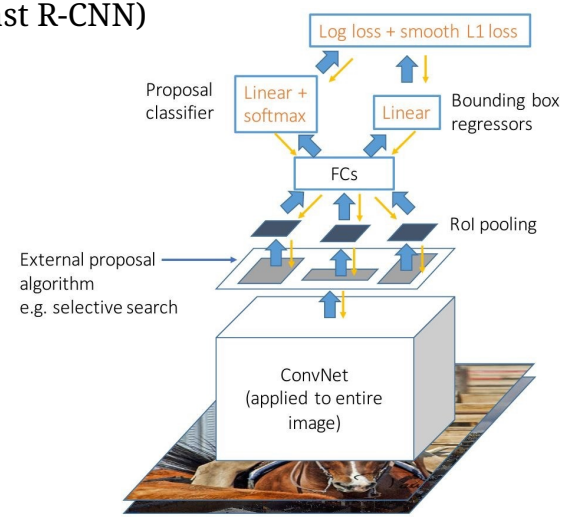
	非常相似数据集	**非常不同数据集**
数据非常少	使用线性分类器作用于顶层	N
相当多 □ □	微调少数几层	N



	非常相似数据集	**非常不同数据集**
数据非常少	使用线性分类器作用于顶层	你就成功了 麻烦……试试线性分类器来自不同阶段
相当多的	微调一个少量层	微调更多层

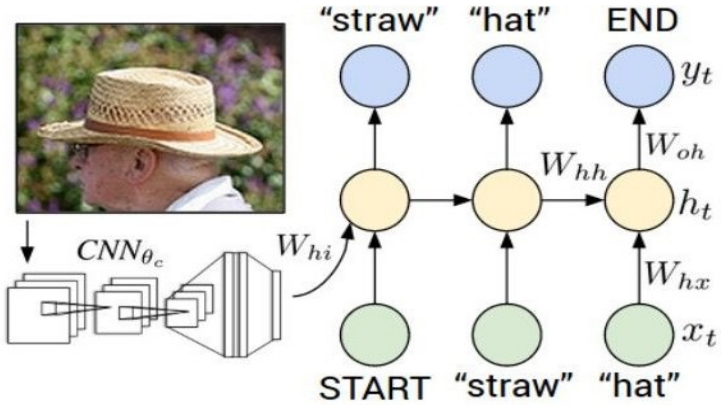
使用CNNs的迁移学习非常普遍... (这是常态，而非例外)

目标检测
(Fast R-CNN)



Girshick, “Fast R-CNN”, ICCV 2015 图版权 Ross Girshick, 2015。经许可复制。

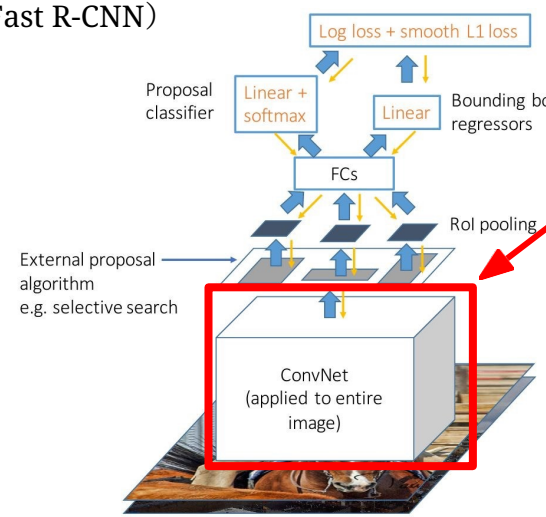
图像描述：CNN + RNN



Karpathy and Fei-Fei, “Deep Visual-Semantic Alignments for Generating Image Descriptions”, CVPR 2015 图版权 IEEE, 2015。为教育目的复制。

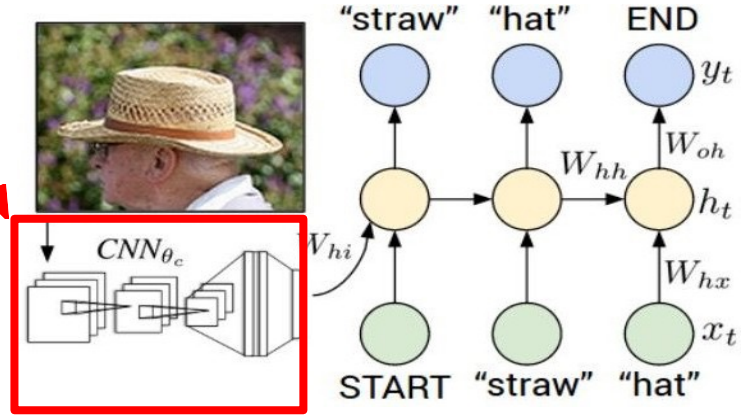
基于 CNNs 的迁移学习已无处不在……（这是常态，而非例外）

目标检测
（Fast R-CNN）



CNN 预训练
在 ImageNet 上

图像描述生成：CNN + RNN

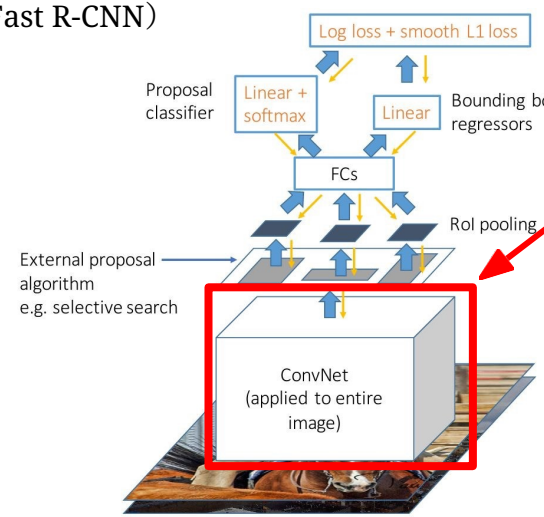


Girshick，“Fast R-CNN”，ICCV 2015 图版权归 Ross Girshick 所有，2015。经许可转载。

Karpathy 和 Fei-Fei，“Deep Visual-Semantic Alignments for Generating Image Descriptions”，CVPR 2015 图版权归 IEEE 所有，2015。出于教育目的复制。

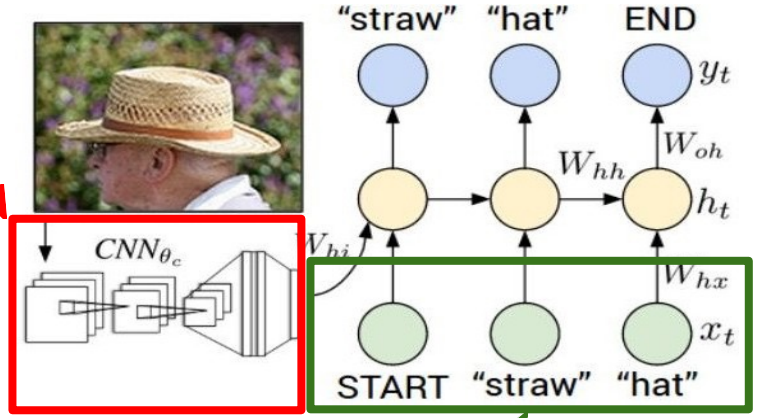
使用 CNNs 的迁移学习十分普遍……（这是常态，而非例外）

目标检测
（Fast R-CNN）



预训练的 CNN
在 ImageNet 上

图像描述：CNN + RNN



预训练的词向量
with word2vec

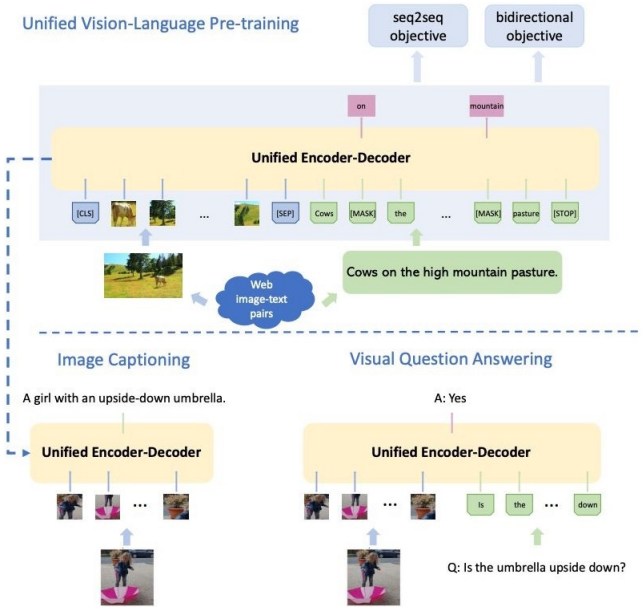
Girshick，"Fast R-CNN"，ICCV 2015
图版权归 Ross Girshick 所有，2015 年。经许可复制。

Karpathy 和 Fei-Fei，"深度视觉-语义对齐用于 Generating Image Descriptions"，CVPR 2015
图版权归 IEEE 所有，2015。为教育目的复制。



基于 CNNs 的迁移学习非常普遍……（这是常态，而非例外）

视觉问答的一个示例



Zhou 等，“面向图像描述与 VQA 的统一视觉-语言预训练” CVPR 2020
图版权 Luowei Zhou，2020。经许可转载。

1. 在ImageNet 上训练 CNN 2. 针对目标检测微调 (1) 3. 在大量文本上训练BERT 语言模型 4. 结合 (2) 和 (3)，训练联合图像 / 语言建模 5. 针对图像描述、视觉问答等微调 (4)。

Krishna 等，“Visual Genome：使用众包密集图像标注连接语言与视觉” IJCV 2017 Devlin 等，“BERT：用于语言理解的深度双向 Transformer 预训练” ArXiv 2018e

匿名课堂反馈

Q&A